

ANKARA YILDIRIM BEYAZIT UNIVERSITY
FACULTY OF ENGINEERING AND NATURAL SCIENCES



Design and Implementation of a Semi-Autonomous RC Car

Graduation Project by

Abdelrahman Hassan

Abdullahi Ayanbukola Ayandiran

Department of Mechanical Engineering

June, 2026

ANKARA

A Graduation Project Submitted to
The Faculty of Engineering and Natural Sciences of
Ankara Yıldırım Beyazıt University
In Partial Fulfillment of the Requirements for the Graduation in Department of
Mechanical Engineering

by
Abdelrahman Hassan
Abdullahi Ayanbukola Ayandiran

June, 2026

ANKARA

ACKNOWLEDGMENTS

Firstly, I would like to express my sincere gratitude to my supervisor, Asst. Prof. TOLGA ÖZASLAN, his tremendous support and motivation during my study. His immense knowledge and precious recommendations constituted the milestones of this study. His guidance assisted me all the time of my research and while writing this graduation project

2026, 24 June

Abdelrahman

Hassan

Abdullahi Ayanbukola Ayandiran

Design and Implementation of a Semi-Autonomous RC Car

ABSTRACT

This project presents the design, manufacturing, and implementation of a one-tenth scale semi-autonomous vehicle inspired by the open-source F1Tenth platform. The primary objective was to build a robust system by optimizing its mechanical geometry, electronic hardware, and embedded software. For the mechanical design, a custom six-bar Ackerman steering linkage was developed to resolve the physical limitations of standard four-bar mechanisms. Using kinematic simulations and surface optimization techniques, the steering error between the inner and outer wheels was significantly minimized. The physical structure incorporates a staggered track width for stability, utilizing high-density polyethylene chassis plates manufactured via subtractive routing, combined with additively manufactured suspension components.

The electronic architecture relies on a custom single-layer printed circuit board designed to segregate power domains, effectively isolating high-current motor noise from sensitive logic sensors. Control is managed by a dual-core microcontroller running a real-time operating system. The software architecture distributes workloads based on timing constraints: one core handles high-level decision making, including remote control signal filtering, Ackermann differential speed calculations, and an ultrasonic obstacle avoidance state machine. The second core is locked into a deterministic one-millisecond loop to execute proportional-integral motor control and dynamic moving average filtering based on feedback from magnetic position encoders. The resulting platform successfully integrates precise mechanical tracking with reliable, interference-free electronic control.

Keywords: Semi-autonomous vehicle, Ackerman steering, kinematic optimization, real-time operating system, printed circuit board, proportional-integral control

Contents

ACKNOWLEDGMENTS.....	iii
Design and Implementation of a Semi-Autonomous RC Car	iv
ABSTRACT	iv
Contents	v
LIST OF TABLES.....	ix
Chapter 1: Mechanical Analysis & Implementation, Design & Manufacturing.....	1
Section 1.1: Introduction	1
Section 1.2: Ackerman Steering Geometry	1
Section 1.2.1: The Ackerman Equation.....	2
Section 1.2.2: The Trapezoidal Equation: [15]	3
Section 1.2.3: Custom Ackerman Steering geometry EQUATION (Ft. DIY).	4
Section 1.2.3: 6-Bar linkage Equation [16].....	4
Section 1.2.4: The MATLAB SIMULATION.....	6
Section 1.2.5.: 4bar linkage – Servo horn constraint.	6
Section 1.3: Optimization	7
Section 1.4.0: Discussion and Comparative Analysis	13
Section 1.5: Mechanical Design and Manufacturing Processes	14
Section 1.5.1: Introduction and General Design	14
Section 1.5.2: Manufacturing Procedure.....	15
Section 1.5.3: List of Fasteners	17
Fastened Parts.....	18
Fasteners.....	18
Description	18
Chapter2: Electronics Design and Implementation	18
2.1 System Overview.....	18
2.2 electronics component	19
2.3 filters.....	20
2.4 circuit design.....	23
2.5 PCB deign.....	24

2.5.1 PCB Layout and Current Capacity	24
2.5.2 CNC Isolation Milling and Assembly	24
Chapter 3: Embedded system.....	26
3.1 System Architecture.....	26
3.1.1 Hardware Integration Layer	26
3.1.2 Software Layer (FreeRTOS Dual-Core Allocation).....	27
3.2 First-Order System Modeling.....	28
Appendix A: Calculations	30
Appendix B: The MATLAB Code.....	31
Appendix C : MATLAB SURFACE OPTOMAZTIO 6-LINK MECHANISM.....	34
Appendix D : EMBEDDED CODE	37
References	52

NOMENCLATURE

Roman Letter Symbols

- L : Wheelbase (car's length)
- W : Track width (car's width)
- d : Steering arm
- L_T : Tie Rod Length
- R_s : Servo link
- KP: King Pivot
- R_p : Maximum allowable pull-up resistance
- R_{min} : Minimum allowable pull-up resistance
- t_r : Rise time of the signal
- C_p : Total bus capacitance
- V_{bus} : Bus supply voltage
- V_{OL} : Output low voltage of the device
- I_{OL} : Output low current (sink current) of the device
- K : system gain
- $u(t)$: control output (PWM)
- $e(t)$: speed error
- K_p : proportional gain
- K_i : integral gain

Greek Letter Symbols

- τ : experimentally identified time constant =0.9s
- δ_i : Inner wheel angle
- δ_o : outer wheel angle
- β : Initial bend angle (Ackerman Angle)
- β_s : Servo angle

Subscripts:

- p: Pull-up / parasitic bus parameter
- min: Minimum value
- bus: Bus-related parameter
- OL: Output low level
- c: Controller design parameter
- BMS : Battery Management System

Acronyms

LIST OF TABLES

Table A: Table 1.1	18
Table B Table2.1: electronics component	20
Table C Table2.2: AS5600 specification.....	22

LIST OF FIGURES

Figure 1.1: ICR of the Ackerman geometry.....	2
Figure 1.2	3
Figure 1.3.....	6
Figure 1.4: initial $R_s = 25\text{mm}$	6
figure 1.5a: 6Link Error Surface	8
figure 1.5b: 4Link Error Surface	8
figure1.6a: comparing outer wheel between mechanism.....	9
figure1.6b: comparing Inner wheel between mechanism.....	9
figure1.7a: comparing inner wheel between grid size.....	10
figure1.7b: comparing outer wheel between grid size.....	10
figure 1.8: Random Search Error distribution.....	11
figure1.9a: comparison of Outer Wheel between iteration.....	12
Figure1.9b: comparison of Inner Wheel between iteration.....	12
Figure1.10a: Comparative Analysis between Random Search Optimization and Grid Search Optimization Outer Wheel.....	13
figure1.10b: Comparative Analysis between Random Search Optimization and Grid Search Optimization Inner Wheel.....	13
Figure 1.11a: Top View; Staggered Car Track.	15
Figure 1.11b: Ackerman Steering Linkage.....	15
Figure 1.12a: Top Chassis.....	15
Figure 1.12b: Bottom Chassis.....	15
Figure 1.13: Exploded View of The Steering System.....	16
Figure 1.14: Rear Drive System	17
Figure 1.15: Exploded View of Final Assembly	17
figure2.1: pull-up resistor.....	21
Figure 2.2: Application of I2C.....	22
Figure 2.3: schematic diagram.....	23
Figure 2.4: PCB diagram.....	25
Figure 3.1: Step Response of DC Motor System.....	29

Chapter 1: Mechanical Analysis & Implementation, Design & Manufacturing.

Section 1.1: Introduction

The development of remote-controlled (RC) cars has significantly evolved, shifting from the usual recreational devices to sophisticated platforms, used to test high-end robotics and automated systems for real-scale cars. The project was inspired by the F1Tenth open-source platform, which provides the software infrastructure to control 1/10-scale cars that are built on ROS2 (Robot Operating system), offering both manual teleoperation and autonomous navigation. The project is aimed at building a high-end RC car, which will be focused on attaining an efficient steering mechanism by hand calculations and simulations (MATLAB), using high definition encoders and motors to integrate an electronic differential, controlling the car through a wireless link using an RC receiver, transmitter and microcontroller(s), implementing obstacle avoidance using ultrasonic sensors, and attaining high traction control on the wheels in order to avoid slips and drags on indoor tiled floors. In addition to that, unlike the regular conventional cars, where the width of the front and rear wheels is the same, a staggered track wheel is integrated. This is to provide a wider base of support, making the car harder to roll over during sharp, high-speed cornering and to accommodate the rear wheel drive & electronic differential setup.

Section 1.2: Ackerman Steering Geometry

Firstly, what is Ackerman Steering Geometry? During a turn, the wheels of the vehicle travel at different radii, which induces scrubbing of the wheel if both wheels move parallel to each other, leading to wear and tear. Therefore, the Ackerman steering geometry is a geometric arrangement that allows a vehicle's front wheels to turn at different radii without fighting each other, by inducing varying steer angles of the inner (at the direction of steer) and outer (opposite the direction of steer) wheels that minimizes wear and tear. The inner wheel being in the direction of the turn will travel an arc of smaller radius to that of the outer, which requires the inner to have a

sharper steer angle than the other [13]. The visual relationship between the steer angles of both wheels is given below:

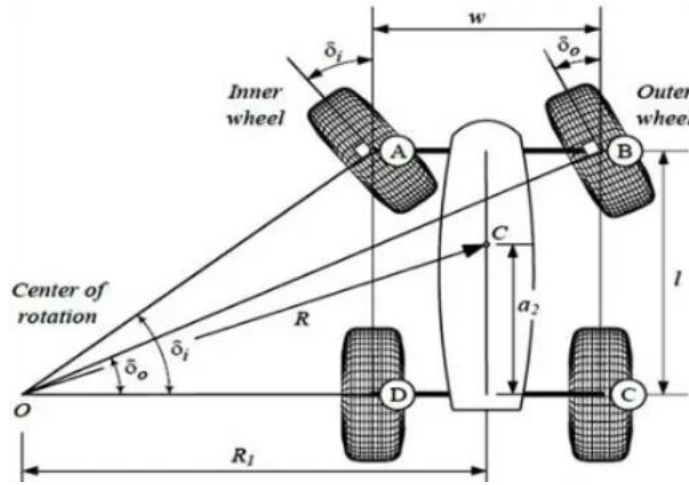


Figure 1.1: ICR of the Ackerman geometry. [18]

As we can see from the above figure, the inner wheel has a sharper steer angle than that of the outer wheel and the steering geometry follows the principle of instantaneous center of rotation(ICR) which dictates that the axle extension of all four wheels must intersect or converge at a point, the IC. However, this is for pure Ackerman, and not the same scenario for the real-life application; this is expressed as percentage of Ackerman (e.g.: a steering could be optimized to meet all needs at 70% Ackerman), which brings about the RMS Error.

Section 1.2.1: The Ackerman Equation

According to the pure Ackerman scenario, the steer angles of the inner and outer wheels are related to each other by the following equations: [14]

$$\tan \delta_o = \frac{(L \times \tan \delta_i)}{L + W \tan \delta_i}, \quad \tan \delta_i = \frac{(L \times \tan \delta_o)}{L - W \tan \delta_o} \quad \text{Equation 1.1.1}$$

$$\beta = \arctan\left(\frac{W}{2L}\right)$$

Section 1.2.2: The Trapezoidal Equation: [15]

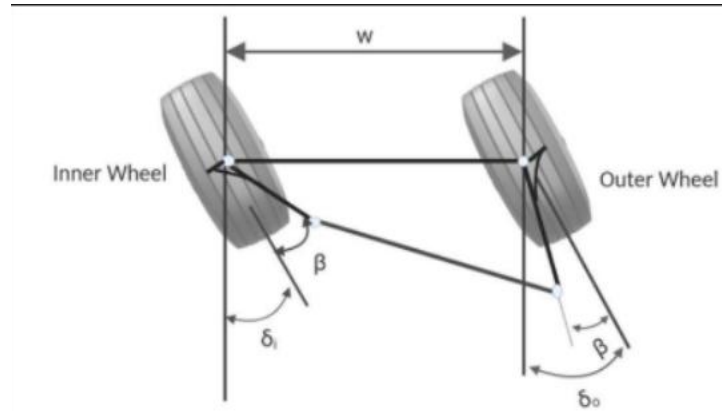


Figure 1.2 [18]

$$\begin{aligned} & \sin(\beta + \delta_i) + \sin(\beta - \delta_o) \\ &= \frac{W}{d} + \sqrt{\left(\left(\frac{W}{d} - 2\sin\beta\right)^2 - (\cos(\beta - \delta_o) - \cos(\beta + \delta_i))^2\right)} \end{aligned}$$

Equation 1.2.1

The Ackerman steering geometry of a Four-Bar linkage takes the shape of a trapezium, which brings about the trapezoidal steering equation as shown in the above figure and equation. The Ackerman equation given in section 1.2.1 is for an ideal case, the perfect Ackerman, while the use of the trapezoidal equation gives the actual angle position of the outer wheel relating to the position of the inner and vice versa. A kinematic analysis of a steering geometry for a 1/10 car will be carried out to show the position effect of the inner wheel to the outer wheel: (Four-Bar linkage).

[Appendix A]

Section 1.2.3: Custom Ackerman Steering geometry EQUATION (Ft. DIY).

The regular **Four-Bar Ackerman** geometry has a limitation to it, and the limitation being the physical constraint of the **Servo horn** (servo motor link). Therefore, to account for this limit, a custom linkage is needed, and this is gotten by simply **splitting** the Tie Rod length into two equal independent halves, making it **5-bar linkage** and including the effect of servo motor link input, a **6-bar linkage**. However, there are other methods to account for this limit, one being changing the **pin joint** between the Tie Rod and the Servo link to a **sliding joint**.

Section 1.2.3: 6-Bar linkage Equation [16]

In this 6-bar linkage, we accounted for the effect of the servo link input and not just the relationship between the inner and outer wheel like the Four-Bar Ackerman linkage. This brings about the analysis of three angle parameters, the inner δ_i , outer δ_o , and the servo link angle β_s . However, calculating the relationship between the three angles by hand will be difficult, which brings about the use of MATLAB to simulate the effect of the servo angle input on the inner and outer wheels. The 6-Bar linkage equation was formulated using the Vector Coordinate equations [16], then with that, a series of MATLAB code was generated. The Equations goes thus:

I. Wheel Pivot Coordinates: King Pivots

$$X_1 = 0, Y_1 = 0, \quad X_2 = W, Y_2 = 0$$

Eq: 1.2.2

II. Left Steering Arm Endpoint:

$$X_3 = d \sin (\beta + \delta_{t1}) \quad \text{Eq: 1.2.3}$$

$$Y_3 = d \cos (\beta + \delta_{t1}) \quad \text{Eq: 1.2.4}$$

$$d_1 = \sqrt{(X_3 - X_1)^2 + (Y_3 - Y_1)^2} \text{ Eq:1.2.5}$$

III. Right Steering Arm Endpoint:

$$X_4 = W - d \sin(\beta - \delta_{t2}) \text{ Eq: 1.2.6}$$

$$Y_4 = d \cos(\beta - \delta_{t2}) \text{ Eq: 1.2.7}$$

$$d_2 = \sqrt{(X_4 - X_3)^2 + (Y_4 - Y_3)^2} \text{ Eq: 1.2.8}$$

IV. Servo Pivot Point and Endpoint:

$$X_5 = \frac{W}{2}, Y_5 = R_s + d \cos(\beta) \text{ Eq: 1.2.9}$$

$$X_6 = \frac{W}{2} + R_s \sin(\beta_s) \text{ Eq: 1.2.10}$$

$$Y_6 = d \cos(\beta) + R_s (1 - \cos(\beta_s)) \text{ Eq: 1.2.11}$$

$$R_s = \sqrt{(X_6 - X_5)^2 + (Y_6 - Y_5)^2} \text{ Eq: 1.2.12}$$

V. Tie Rod Lengths:

$$L_{T1} = \sqrt{(X_6 - X_3)^2 + (Y_6 - Y_3)^2} \text{ Eq: 1.2.13}$$

$$L_{T2} = \sqrt{(X_6 - X_4)^2 + (Y_6 - Y_4)^2} \text{ Eq: 1.2.14}$$

VI. Defined Constants:

d=22.76mm, R_s=25mm, L_{T1}=L_{T2}=74.46mm, β=15.97°, L=286.52mm, W=164mm

Where R_s is the Servo link length.

Section 1.2.4: The MATLAB SIMULATION.

Using equation 1.2.1 to 1.2.14, a code to run the simulation was created, and below is the graphical representation:

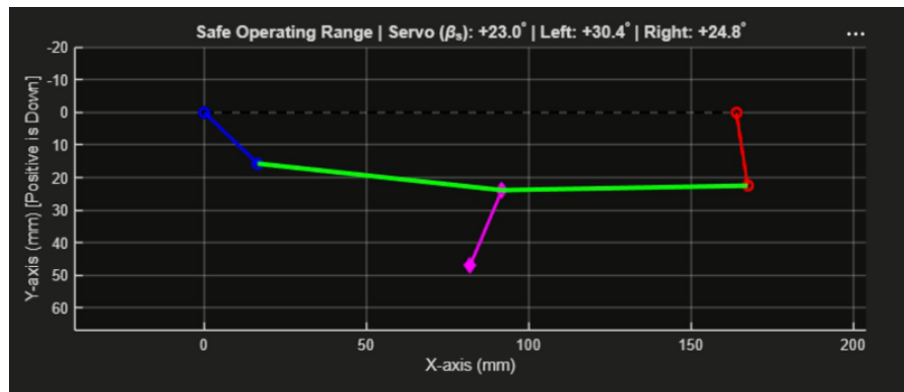


Figure 1.3

From figure 1.3, we could see the relation between the servo input angle, the inner and outer wheel angle. At approximately 30 deg of the inner wheel, we could see the outer wheel angle resulting to 24.8 deg, a value different from the 4bar linkage. This shows that splitting the Tie rods and adding the effect of the servo link could alter the relationship between inner and outer wheels.

Section 1.2.5.: 4bar linkage – Servo horn constraint.

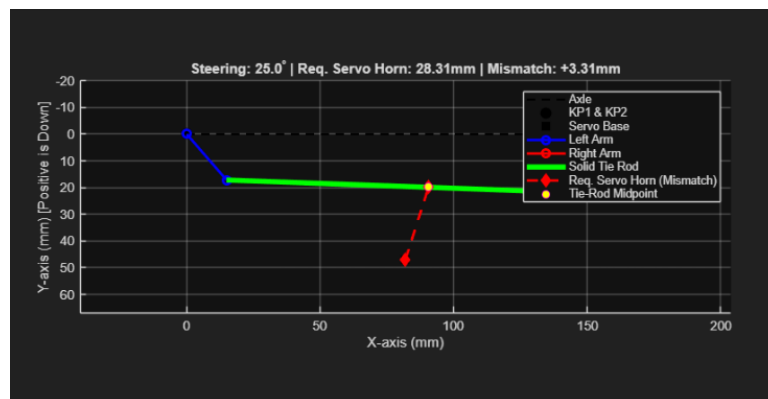


Figure 1.4: initial $R_s = 25\text{mm}$

Figure 1.4 shows the graphical representation of the physical constraint discussed in **section 1.2.3**. As shown in the figure above, we could see that the servo motor link (servo horn) needs to have a constant change in length as the servo angle sweeps just to be in contact with the whole linkage which is realistically not possible. Therefore, splitting the Tie rod was done to overcome that constraint and lead to the custom equation.

Section 1.3: Optimization

Optimization of any system requires defining optimized variables, aim of optimization, Design of Experiments (DOE), as well as Optimization Method (OP) [2]. There are many DOE and OP methods utilized. The objective of optimization is to achieve similarity between the ideal case (Ackermann) and the actual steering system. In this study, two systems are considered: the 4-link trapezoidal steering system and the 6-link steering system, as well as the systems before and after optimization.

Section 1.3.1: Methodology and Design of Experiments (DOE)

Since high sensitivity requirements are associated with the adoption of a limited operating range, the Full Factorial Design method was utilized for testing every possible combination of the variables [2].

Section 1.3.2: Objective Function

Therefore, the expression of error is defined by the difference between the turning angle of the inner wheel as well as the outer wheel of the ideal case and both wheels for the actual case [9]. The RMSE is used as the objective function to give more weight to larger errors due to the squaring operation, which minimizes the overall error result [3].

$$e_{ESM} = \left(\frac{1}{n} \sum_{i=1}^n (\delta_{actual} - \delta_{Ackermann})^2 \right)^{0.5} \text{ Equation 1.3.1}$$

Where one angle is considered as input and the other wheels angle as the output to evaluate the steering system response [9].

Section 1.3.3: Variables and Constraints

The parameter range is (10–30) for the β and (2–30) for Steering Arm Length L , with a step size of 0.3 for each operation. Choosing the step size includes a trade-off with round-off error; an incorrect step size may cause the solution to diverge or fail to converge the optimal minimum error [1].

A 3D surface plot is utilized by defining the equations in MATLAB to visualize the characteristic change of β and L with RMSE error, which has radian units (figure 1.5a) (figure 1.5b).

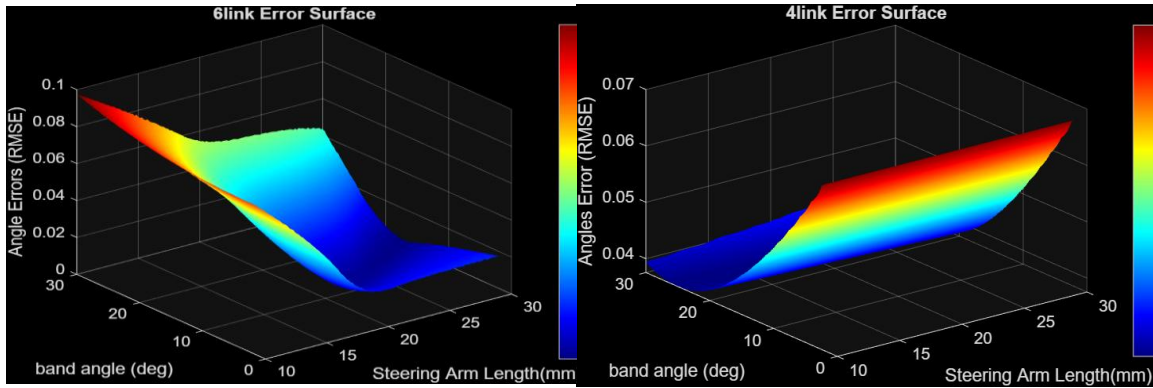


figure 1.5a: 6 Link Error Surface

figure 1.5b: 6 Link Error Surface

In addition, the selected ranges are specified by the following conditions: a longer arm length significantly increases the sweep radius, leading to mechanical interference with vehicle systems, as well as slower steering response due to increasing the turning radius [4][8]. Additionally, setting β greater than 30° makes the system reach a kinematic singularity. At some point, the mechanism starts acting as an uncontrolled system, and the servo motor will never be able to control it back, even if the applied torque is too large [5][4].

Section 1.3.4: Optimization Algorithms and Results

1-Grid Search Optimization

A Grid Search algorithm is used to evaluate all candidates in a discrete set and select the solution with minimum RMSE [7]. The results of the surface optimization show the following: the best values of β and d are (24.7° , 10.45 mm) for the 4-link trapezoidal



figure1.6a: comparing outer wheel between mechanism



figure1.6b: comparing Inner wheel between mechanism

steering system (figure 1.5a), and (15.65°, 28.45 mm) for the 6-link steering system (figure 1.5b). The initial parameters for the 6-link mechanism before optimization were (15.97°, 22.76 mm). For comparison, equations from 1.1 to 2.2 were utilized in Excel to evaluate the kinematic behavior plot for wheel angles. The comparative results are presented in figure1.3.3 and figure1.3.4 using the same error criteria (Equation 1.3.1).

As shown in figure1.6b and figure1.6b, there is divergence of the 4-link system and convergence for the optimized 6-link system. Therefore, the study will continue trying to converge to the best values for the studied parameters by changing the grid size of the grid search and by using another search method. The grid size for the input was selected as 0.3, while the wheel angle search space grid size was also changed to observe the characteristics of the optimization. The selected size was 0.5, by taking values before and after this point to determine whether the solution diverges or converges toward the minimum error.

For a grid size of 0.4, the obtained values were:

- $\beta = 17.30^\circ$
- $d = 29.95 \text{ mm}$

For a grid size of 0.2, the obtained values were:

- $\beta: = 17.15^\circ$
- d: 29.80 mm

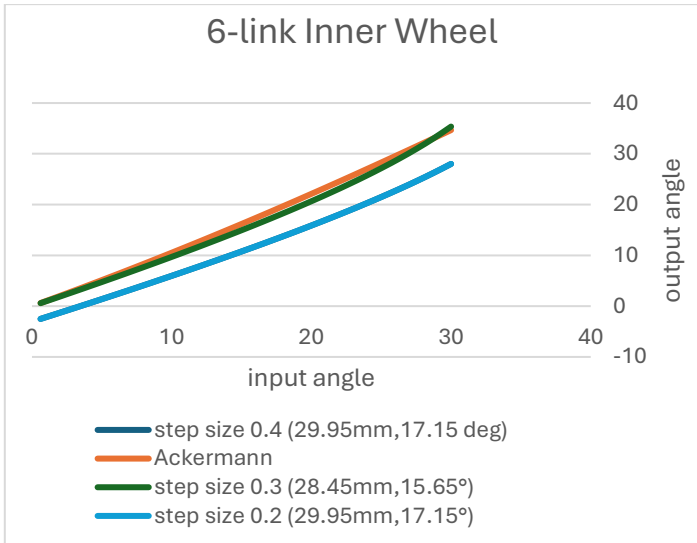


figure1.7a: comparing inner wheel between grid size

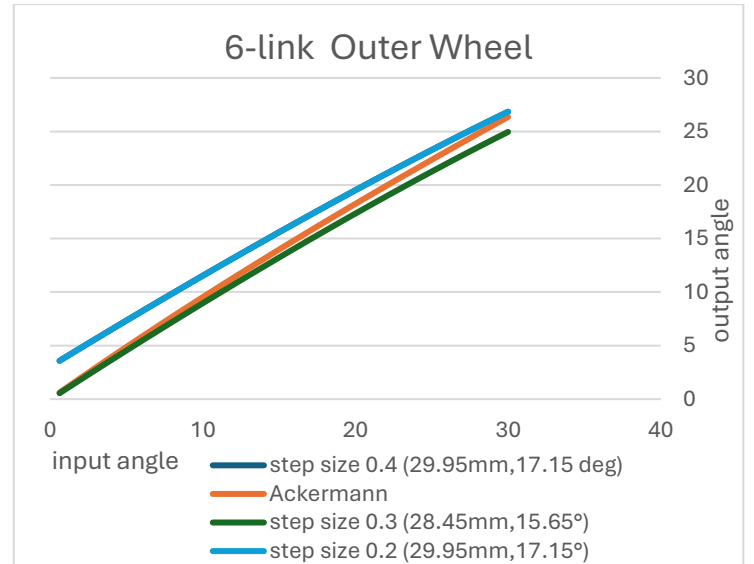


figure1.7b: comparing outer wheel between grid size

Afterward, both the wheel angle search space and the input were assigned the best grid sizes that produced the least error, which were 0.3 for the input angle, as shown in Figure 1.7a and Figure 1.7b. The step size of 0.3 showed the highest similarity with the Ackermann system; therefore, it was selected as the best parameter for this search.

2-Random Search Optimization

To validate the grid search results and provide a stochastic comparison, Random Search optimization was applied [7] to verify whether the system had reached the optimal values. Unlike Grid Search, which evaluates predefined discrete points, Random Search explores the parameter space randomly, improving the chance of reaching a global optimum.

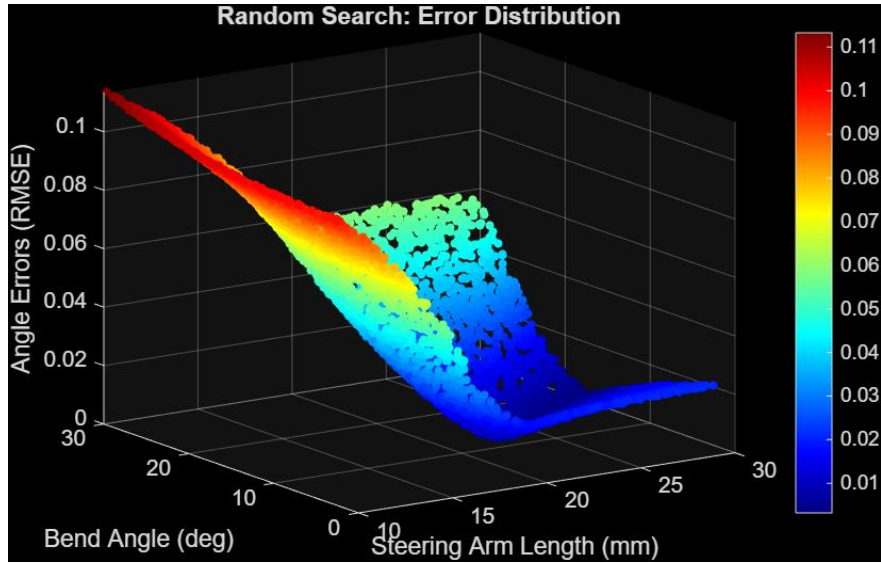


figure 1.8: Random Search Error distribution

As in the grid search, this method also requires a specified number of iterations to obtain the optimal solution. Therefore, three different iteration numbers were used to observe the behavior of the results and determine whether they diverge or converge toward the optimal value. A parametric study was conducted using different iteration counts ($N_{\text{iteration}} = 500, 5000, 50000$) to evaluate their effect on convergence behavior.

For:

- $N_{\text{iteration}} = 5000$

The results were:

- $\beta: 15.79^\circ$
- $d: 28.52 \text{ mm}$

For:

- $N_{\text{iteration}} = 50000$

The results were:

- $\beta: 17.54^\circ$
- $d: 29.78 \text{ mm}$

For:

- $N_{\text{iteration}} = 500$

The results were:

- $\beta 16.43^\circ$
- $d: 29.56 \text{ mm}$

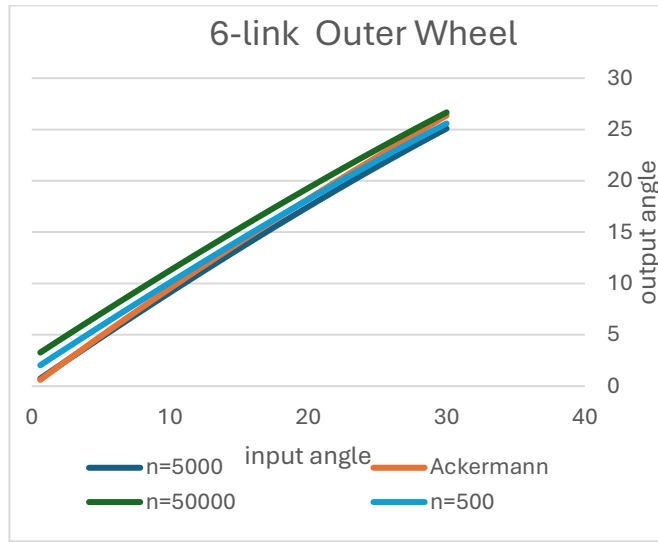


figure1.9a: comparison of Outer Wheel between iteration

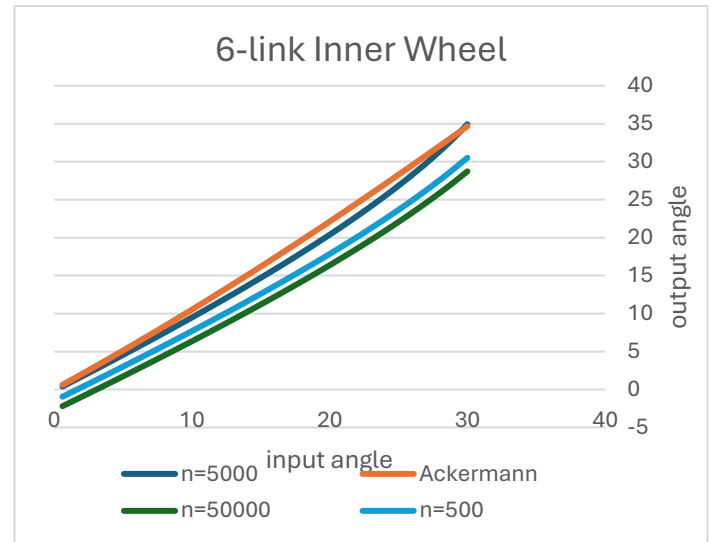


figure1.9b: comparison of Inner Wheel between iteration

As shown in Figure 1.9b, the best and most stable performance is obtained at $N_{\text{iteration}} = 5000$ due to convergence toward the ideal system.

Comparative Analysis between Random Search Optimization and Grid Search Optimization

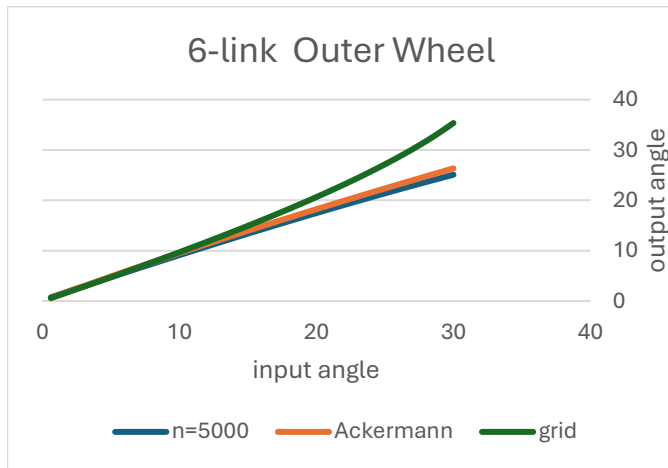


figure 1.10a: Comparative Analysis between Random Search Optimization and Grid Search Optimization Outer Wheel

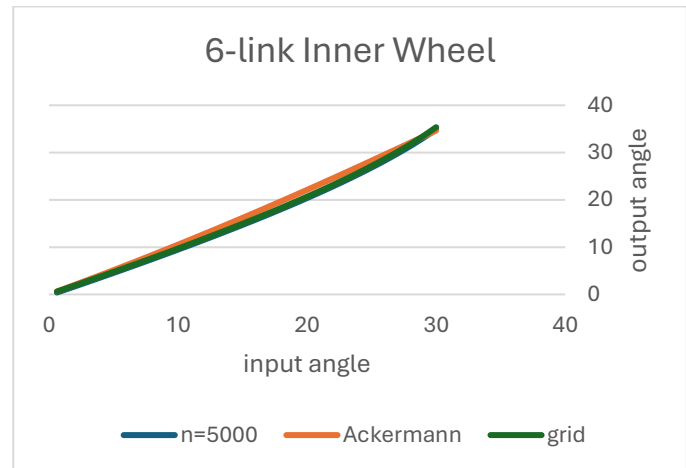


figure1.10b: Comparative Analysis between Random Search Optimization and Grid Search Optimization Inner Wheel

As figure 1.10a and figure 1.10b shows, the inner wheel for both methods gives similar behavior when compared with the Ackermann system. However, random search behaves better than grid search in the high-angle range due to its stochastic sampling, which allows exploration of a wider solution space compared to the discrete nature of grid search. The grid search solution starts to diverge from the ideal case as the angle increases.

Section 1.4.0: Discussion and Comparative Analysis

As previously mentioned, the actual systems will never have zero error because the actual systems can match the ideal Ackermann system only at a few points and cannot fully match it [6][9]. Consequently, the plot shows the characteristics of the systems over a range of 30 degrees of the input (the servo motor), where the target was 25 degrees.

Furthermore, the optimized 6-link system shows the best match to the ideal case, along with a relatively smoother response compared to before optimization. Accordingly, the system has a linear relationship with a small slope value, which indicates lower system sensitivity [10]. Additionally, it has symmetrical solutions for the same range under

negative conditions [11]. Conversely, the 4-link mechanism has a nonlinear and more complex relationship for the inner wheel, leading to divergence of both wheels from the ideal case; thus, a clear slope cannot be defined. Finally, when comparing the error values, the optimized 4-link system has an error of 0.036 radian compared with the 6-link system that has an error of 0.015 radian, whereas before optimization its error was equal to 0.0306 radian. This confirms that increasing the number of links improves the system's ability to approximate the Ackermann steering geometry.

Section 1.5: Mechanical Design and Manufacturing Processes

Section 1.5.1: Introduction and General Design

The primary objective of this project was to design and manufacture a fully functional RC car optimized for stability and directional control. The entire mechanical design was modeled using **SolidWorks** to ensure precise tolerances and accurate assembly before physical production began.

The vehicle operates on a rear-wheel-drive (RWD) system powered by two independent DC motors, providing robust forward propulsion. Directional control is handled by an independent front steering system driven by a servo motor. A key feature of the vehicle's geometry is its **staggered track design**—featuring a wider wheelbase at the rear compared to the front. This design choice was implemented primarily to accommodate the dual rear-motor drive system, while also improving the vehicle's overall traction and stability during acceleration. In addition, the design of the steering system was done in accordance with the **steering system analysis and optimization (section 1.2-1.3)**

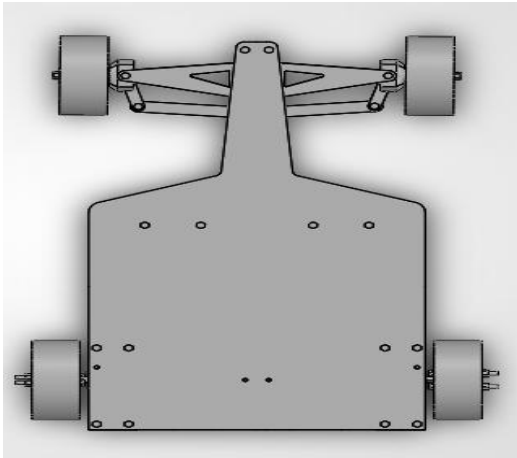


Figure 1.11a: Top View; Staggered Car Track.

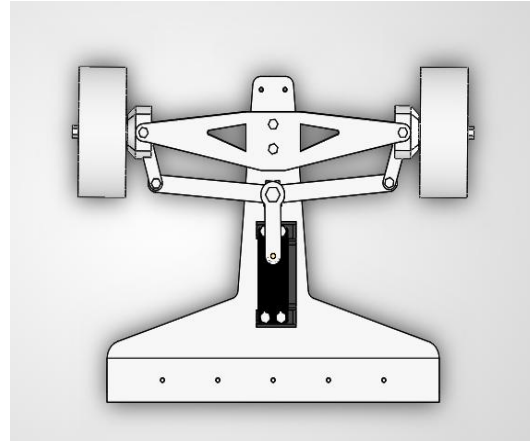


Figure 1.11b: Ackerman Steering Linkage.

Section 1.5.2: Manufacturing Procedure

The physical construction of the RC car was divided into distinct manufacturing and assembly phases, utilizing both subtractive and additive manufacturing techniques.

- I. **Chassis Fabrication (CNC Milling):** The structural foundation of the car consists of a top and bottom chassis. To ensure durability and rigidity, both chassis plates were manufactured from High-Density Polyethylene (HDPE) plastic. A CNC milling machine was used to precisely cut the HDPE sheets to the exact dimensions specified in the SolidWorks CAD model, including all necessary mounting holes and cutouts.

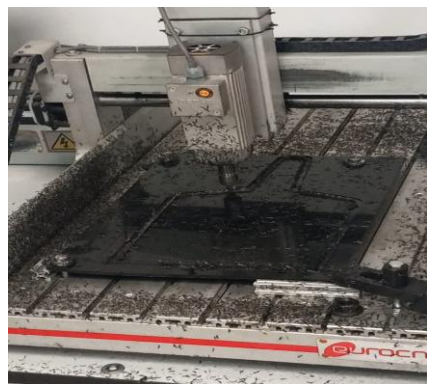


Figure 1.12a: Top Chassis.

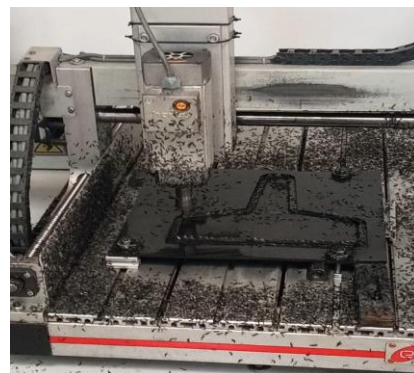


Figure 1.12b: Bottom Chassis.

- II. **Front Steering System Assembly:**

Except for the lower HDPE chassis plate, all components of the front steering and suspension system were 3D printed. Because the front wheels are undriven, the design focuses heavily on smooth rotation and structural integrity.

- **Bearing Assembly:** Each front wheel utilizes two bearings to minimize rotational friction.
- **Shaft Reinforcement:** The assembly follows a specific sequence: wheel hub → front wheel shaft → inner bearing → wheel → outer bearing. An M3x50 bolt is passed through this entire assembly to fasten it together. Crucially, this long bolt acts as an internal metal reinforcement for the 3D-printed front wheel shaft, preventing shearing under load.

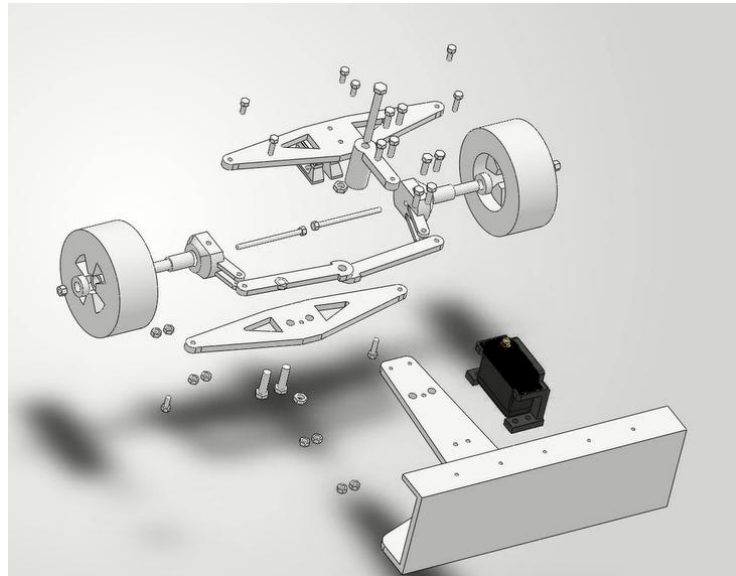


Figure 1.13: Exploded View of The Steering System

- III. **Rear Drive System Assembly:** The rear assembly houses the primary propulsion system. Custom motor mounts were fabricated to secure the two DC motors to the chassis.

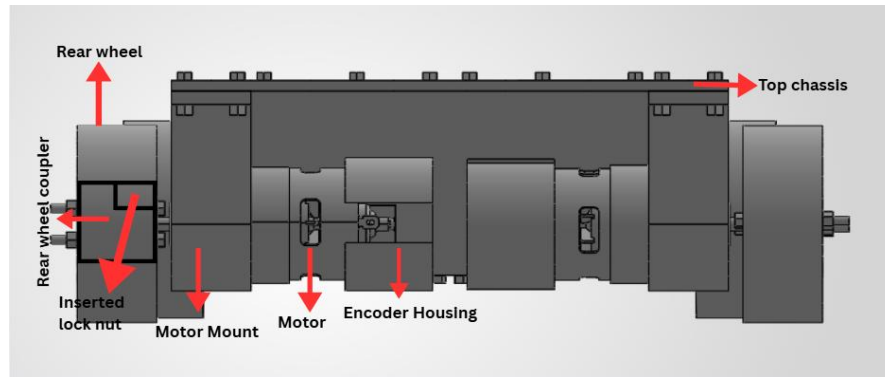


Figure 1.14: Rear Drive System

- IV. **Power Transmission:** To transfer power from the motors to the wheels, custom wheel couplers were utilized. Each coupler features an inserted internal nut, allowing it to act as a set-screw mechanism that fastens tightly against the motor's drive shaft.
- V. **Wheel Mounting:** Once the couplers were secured to the motors, each rear wheel was fastened directly to its respective coupler using two M3x40 screws + nuts, ensuring a rigid connection with no slip.
- VI. **Final Assembly:** The final stage involved bringing all sub-assemblies together. The 3D-printed front steering block and rear motor blocks were mounted to the lower HDP chassis. The dual DC motors and front steering servo were wired and secured, and the top HDPE chassis plate was fastened over the top of the central body to enclose the internal electronics, resulting in a rigid, unified vehicle structure.

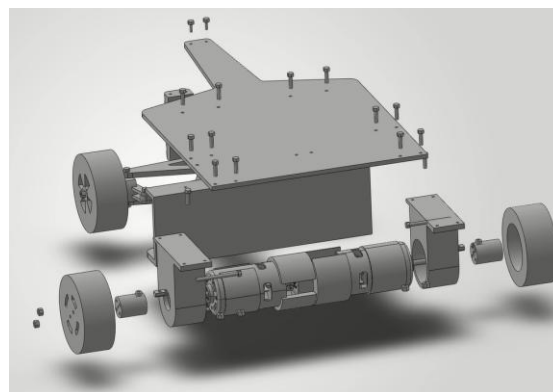


Figure 1.15: Exploded View of Final Assembly.

Section 1.5.3: List of Fasteners

Fastened Parts	Fasteners	Description
Wheel hubs – A-arms	4x ‘M3 x 8’ Countersunk Screws	
Arms Block – Bottom A-arms – Bottom Chassis	2x ‘M3 x 12’ Countersunk Screws + 2x M3 nuts	
Wheel hubs – Front shaft – Inner bearing – wheel – Outer bearing	2x ‘M3 x 50’ Bolts + 2x ‘M3 Nuts’	
Top A-arms – Arms Block	2x ‘M3 x 8’ self-drill flathead screws	
Wheels hubs – Tie rods	2x ‘M3 x 8’ Countersunk Screws	
Servo link – Tie rods	1x ‘M5 x 40’ Bolt + 1x ‘M5 Nut’	
Top chassis – Chasses connector – Bottom chassis	4x ‘M3 x 8’ self-drill flathead screws	Front Connection
Top chassis – Bottom chassis	8x ‘M3 x 15’ Countersunk Screws + 8x M3 nuts	Center connection (of top chassis)
Rear wheel coupler – Motor shaft	2x ‘M3 x 12’ Countersunk Screws + 2x inserted square nuts.	The coupler is fastened to the motor shaft through the inserted nut along the screw’s path to have a rigid connection
Rear wheel coupler – Rear wheel	4x ‘M3 x 35’ Countersunk Screws + 4x ‘M3 Nuts’	
Motor Mount – Top chassis	8x ‘M3 x 15’ Countersunk Screws + 8x ‘M3 Nuts’	

Table A: Table 1.1

Chapter2: Electronics Design and Implementation

2.1 System Overview

In this section, we will dive into the fundamentals of electronics for our semi-automatic RC car. We will focus on selecting electronic components, filters, and I2C communication, circuit design, PCB design, to achieve a good system response, as well as developing an efficient actuator system.

2.2 electronics component

electronics	voltage	current	information
Two DC motors	12 V	6 A	3000 RPM [26].
Two BTS7960 IBT_2 motor drive module	(12-24) V 5 V driver circuit		The driver controls DC motor speed and direction through RPWM and LPWM signals from the microcontroller. One signal is driving HIGH with PWM to set the rotation direction, while the duty cycle controls the motor speed. R_EN and L_EN are both set HIGH (5V) to enable CW and CCW operation.[27]
AS5600 magnetic encoders	(3.3V–5V).	100mA	The encoder was powered at 3.3V from the ESP32 for safe communication. It measures motor shaft position and converts it into speed using the sampling time and its 4023 CPR resolution. Communication with the ESP32 is done through the I²C bus using SCL and SDA lines [18]
Three ultrasonic sensors	5V		The ultrasonic sensor measures distance by sending ultrasonic waves through the TRIG pin and receiving the reflected signal through the ECHO pin. The distance is calculated from the time delay between transmission and reception. It operates with a 5V supply and common ground, with very low current consumption.[19]
Servo motor	5V	0.3-2A	It consists of gears driven by a small DC motor and controlled by a PWM signal to achieve a specific angular position. The pulse width determines the required steering angle, while the internal feedback system adjusts the motor until the target position is reached. It is used to control the steering system.[20]
Dc converters			[22]

Multiplexer (TCA9548A)	5v		The AS5600 uses I²C communication , and since multiple sensors share the same address, a TCA9548A I²C multiplexer is used to separate them. It switches the SCL and SDA channels, allowing the microcontroller to access each sensor individually. Its low power consumption makes it suitable for embedded systems.
FS-iA6B Receiver	5v		[23]
Power supply	12v	10A	3S2P 18650 with 2C have BMS 20A , 12V 3S

Table B Table2.1: electronics component

2.3 filters

2.3.1 Electromagnetic Effects and Filtering

During testing, instability was observed due to motor back-EMF and switching transients, which introduced noise into the system.

To mitigate this:

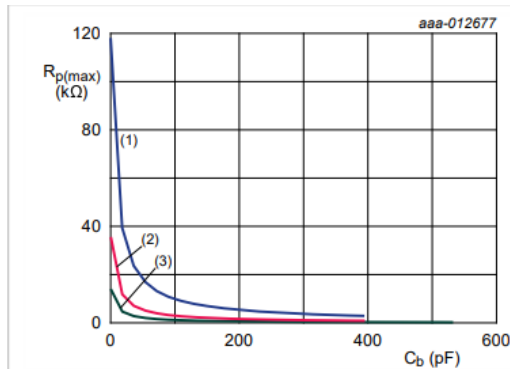
- 0.1 μF ceramic capacitors were placed on serial connection with the motor driver terminals as well as the servo motor to suppress high-frequency noise and reduce voltage spikes.
- 220 μF electrolytic capacitors were added for the receiver to ensure a more stable response and to smooth sudden voltage fluctuations.
- 220 μF electrolytic capacitors were also used for the servo motor to improve stability and handle current spikes during load changes.
- Pull-up resistors were used on the I²C lines to ensure proper signal levels and communication stability [19].
- Software filtering was applied to encoder and RC signals to reduce noise and improve measurement accuracy. These measures improved system stability significantly.

2.3.2 I2C communication

For communication between microcontrollers and multiple devices, an I2C protocol is provided that independently addresses peripherals utilizing just two buses (SCL) and (SDA). The data transfers in bit form. The SCL rising edge forms a 7–8 bit address followed by commands from the master (microcontroller), allowing the addressed peripherals to execute commands and return data with the same clock timing. The physical bus employs an open-drain structure where devices sink current, which means it can make signals in low form. A pull-up resistor is used to make the signals high; for example, the AS5600 has a transistor, and when the order needs to be high, it lets the resistor work to pull the signals up. This design helps to avoid short circuits during communication. To decrease the EMF, an external pull-up resistor is added, which connects in parallel with the internal one, and Equation 2.3.2 is utilized to get the value of the resistor. [19] To counteract electromagnetic noise and support high data rates, external pull-up resistors are often added, which connect in parallel with internal microcontroller resistors to lower the equivalent bus resistance [19]

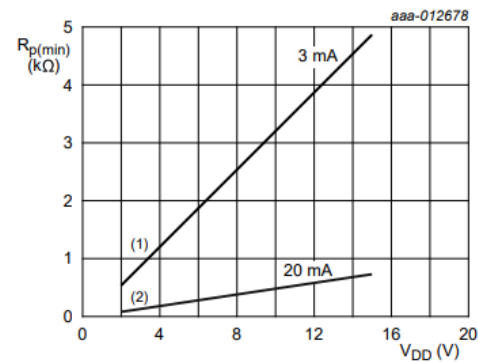
$$R_{eq} = \frac{R_{int} \times R_{ext}}{R_{int} + R_{ext}} \quad \text{Equation 2.3.2}$$

To choose our external resistor value there is two ways first one defined by standard value using data sheet second way calculating it by Equation 2.3.3, Equation 2.3.4, as well as graph [26]



1. Standard-mode
2. Fast-mode
3. Fast-mode Plus

Figure 41. $R_{p(max)}$ as a function of bus capacitance



1. Fast-mode and Standard-mode
2. Fast-mode Plus

Figure 42. $R_{p(min)}$ as a function of V_{DD}

figure2.1: pull-up resistor

$$R_p(max) = \frac{t_r}{0.8473 \times C_p} \quad \text{Equation 2.3.3}$$

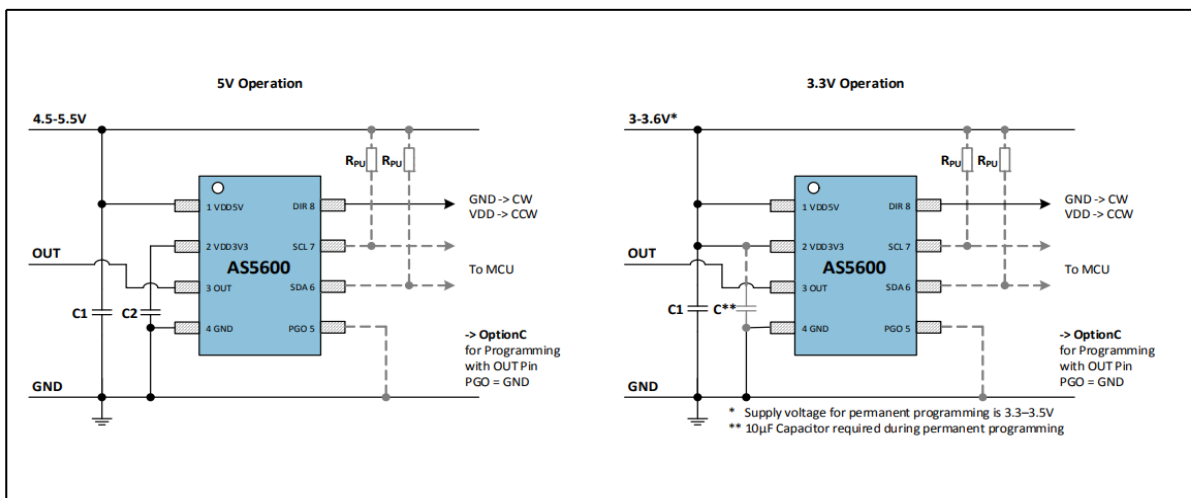
$$R_{min} = \frac{V_{bus} - V_{OL}}{I_{OL}} \quad \text{Equation 2.3.4}$$

	V_{OL}	I_{OL}	C_p	R (pull-up-standard)	address
AS5600	max 0.4 V	20 mA-3mA	550 pF	4.7 k Ω	0x36

Table C Table2.2: AS5600 specification

In our case, an incorrect configuration was installed into the system by ignoring the AS5600 data sheet [1], putting the $V_{in} = 3.3$ V of the encoder different from the one used for the pull-up resistor $V = 5$ V, as well as ignoring the standard resistor value from the data sheet [1], which is 4.7 k Ω , and putting it at 10 k Ω . This mismatch between the external pull-up and the internal one created a fault condition for both AS5600 as well as TCA9548A, which occurred due to exceeding the maximum required voltage for the external pull-up that was calculated by equation 2.3.3, which is equal to 3.6 V for $V_{in} = 3.3$ V. This case caused damage to the transistor, leading to failure of the electronics on the encoder; by that, a high current occurred on the bus connected to TCA9548A and damaged its components. The following figure shows the operation of the circuit with the external resistor [18].

Figure 38:
Application Diagram for Angle Readout and Programming with I²C (Option A and Option C)

Figure 2.2: Application of I²C

2.4 circuit design

The system is powered using a 12 V, 6 A power supply. To decrease noise and voltage fluctuations, the power distribution structure is strictly segregated into three primary domains:

- Actuator domain: 5 V for both TB6612FNG drivers with servo motors via one of the DC-DC converters.
- 5 V domain: 5 V to power the ESP32, TCA9548A multiplexer, ultrasonics as well as the receiver.
- 3.3 V domain: used to power the AS5600.

The ESP32's GPIO pins include protection to prevent overvoltage damage, and diodes, capacitors, and external pull-up resistors are minimized and used to reduce Back-EMF transient spikes from motors and optimize response time without corrupting sensor data.

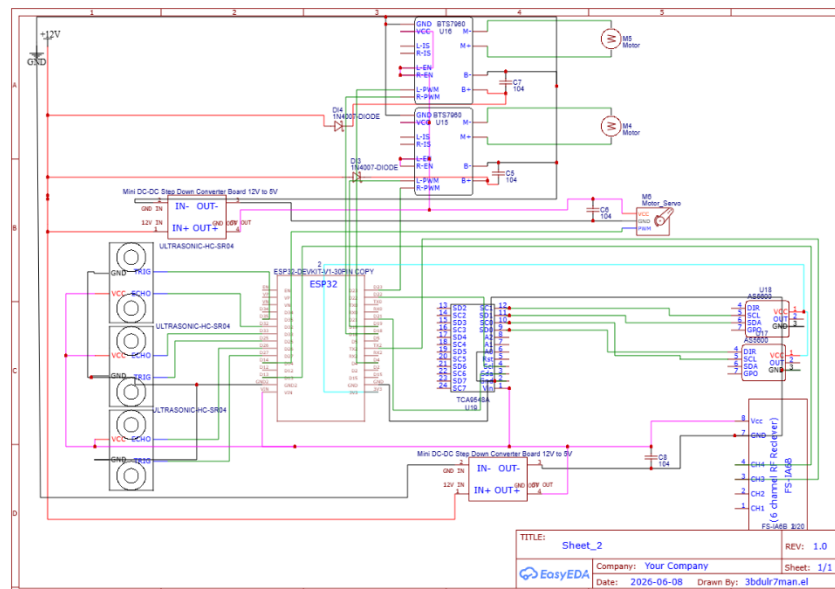


Figure 2.3: schematic diagram

2.5 PCB design

2.5.1 PCB Layout and Current Capacity

The printed circuit board (PCB) was intended for fast in-Lab prototyping purposes. The schematic below shows the circuit structure that is specifically tailored to limit crossovers. To ensure the PCB could be manufactured using desktop CNC technology with sufficient precision, the single-layer approach had to be adopted. It should be noted that several elements labeled "resistors" in the vicinity of ESP32, the multiplexer, and ultrasonic devices in the circuit are not resistors; they are zero-ohm links (or wire jumpers). As such, they allow bridging intersecting traces and implementing pure single-layer design. All the rest of unavoidable crossovers have been addressed by bottom-side wire routing.

Trace widths were carefully calculated considering current carrying properties and heat dissipation capabilities. In order to transfer up to 2A, power lines must be at least 1 mm wide. Otherwise, overheating would lead to degradation of the metal, thereby damaging the traces and causing open circuits. In addition, all the traces associated with power input should be widened as much as possible (polygon pours), because they are expected to provide low impedance to current flow. Screw terminals and female pin headers will provide flexibility in mounting electronic components to the PCB.

2.5.2 CNC Isolation Milling and Assembly

The manufacturing of the actual circuit board involved the use of a CNC 3018 machine to cut out the design on the copper clad laminate material using the isolation milling technique. The physical board had some surface warpage, and therefore, before the milling process took place, it was necessary to use a physical probe in order to get the topography heights of the board for consistent cutting through the entire board.

Computer-Aided Manufacturing (CAM) operations involved two processes, namely:

- Hole Drilling: A 1.0 mm drill diameter was employed to create holes for the THT technology. In addition, a 2.5 mm drill diameter was applied to create the mounting holes that were needed to mount the PCB to the car chassis.
- Milling: The trace isolation milling was done by applying a 60-degree V-bit (3mm) with a plunge depth of 0.9mm.

After the manufacturing process, the circuit board was manually cleaned to rid it of copper waste. The entire soldering procedure was done with precise accuracy, at a steady temperature of 300°C, ensuring robust inter-metallic bonding without causing the pads to peel off due to high heat exposure. Quality control was carried out alongside the assembly stage. A multimeter was employed during each step of the soldering process to carry out rigorous continuity tests, ensuring there was zero electrical connection between the conductive traces and the remaining copper layer on the board.

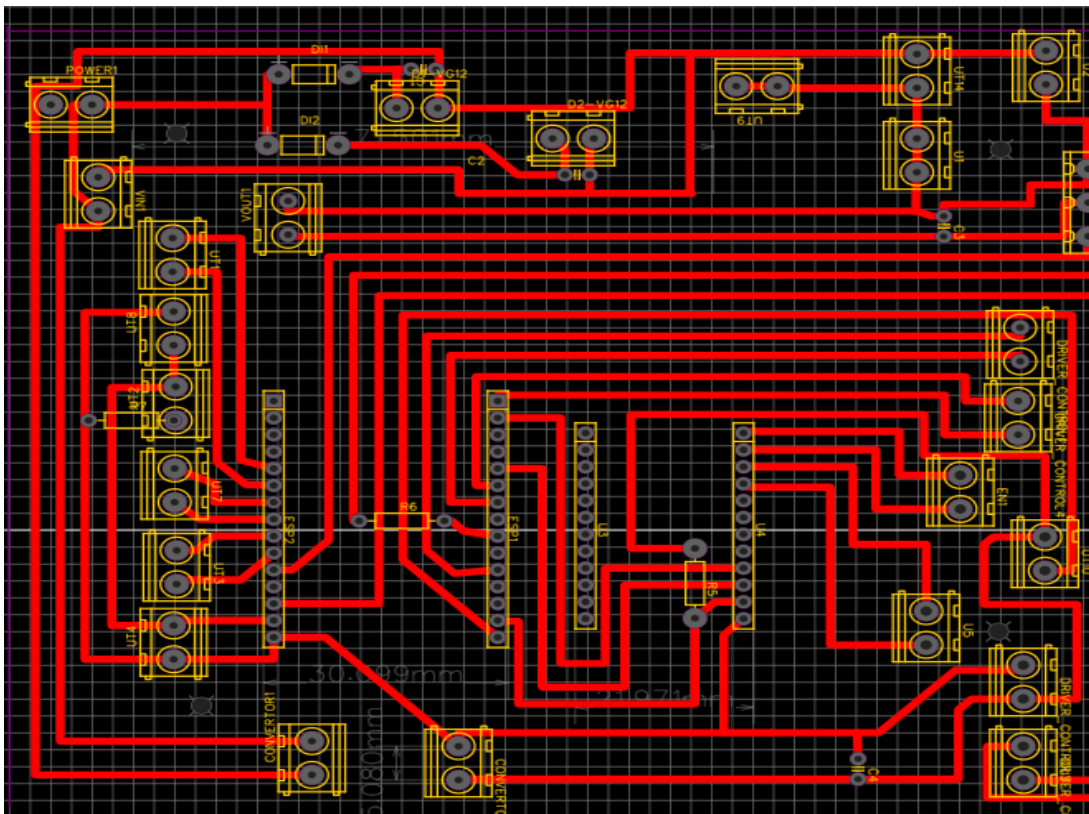


Figure 2.4: PCB diagram

Chapter 3: Embedded system

3.1 System Architecture

The car runs on a distributed hierarchical architecture powered by an ESP32 controller. The idea behind the design is to process asynchronous environmental sensing, user inputs, and high-rate closed-loop control for motors concurrently without any performance bottlenecks.

The architecture comprises three primary components, namely Hardware Integration, FreeRTOS Task Allocation, and Data Parallelism.

3.1.1 Hardware Integration Layer

The hardware topology aims to strictly decouple the steering mechanism from the drive mechanism, thus guaranteeing that pure control signals are achieved.

- Control Input (RC Receiver):
 - o The control algorithm for the input logic is completely isolated on two different interrupts:
 - o Channel 1 (CH1): Is dedicated purely to the steering servo.
 - o Channel 2 (CH2): Is dedicated purely to the longitudinal velocity (DC motors).
- Sensor Inputs:
 - o Ultrasonics: Time of flight ultrasonic sensors allow for obstacle detection. The sensor readings are collected through custom non-blocking logic to ensure that the hardware timeout does not halt the program.
 - o Telemetry: AS5600 magnetic sensors are employed to obtain the speed values of the wheel. Since all the AS5600 have fixed I2C addresses, a TCA9548A I2C multiplexer must be used to avoid bus conflicts.

- Actuators:
 - o An encoder motor driver acts on the PID output, generating a PWM output for the two wheels.
 - o A servomotor acts on the front wheel angular position in degrees mapped to the range of $[60^{\circ}-120^{\circ}]$.

3.1.2 Software Layer (FreeRTOS Dual-Core Allocation)

In order to perform true real-time deterministic control, the ESP32 implements a FreeRTOS dual core configuration. The tasks are divided according to their time execution needs:

Core 1 –Decision, Perception and Kinematics (The "Brain")

Core 1 is the higher-level manager which executes tasks asynchronously and contains all logic that does not require microsecond precision.

1. RC Signal Filtering: Performs signal processing on the raw PWM received from the two channels CH1 and CH2, then maps it to a valid set of values within physical constraints (Steering Angle and Target RPM).
2. State Machine for ADAS: Periodically scans the ultrasonic array in order to detect the presence of any obstacles, determines the mode of operation of the car (NORMAL_RC, AVOIDANCE_OVERRIDE, BRAKING_SEQUENCE or SYSTEM_HALTED) and overrides the RC input in case of any obstacles.
3. Ackermann Wheel Speed Calculations: Takes the basic target RPM value from CH2 and the steering angle value from CH1, calculates the difference between the left and right wheel speeds for cornering.

Core 0 – Deterministic Control Loop (The "Spine")

Core 0 is fixed to an unvarying 1-millisecond hardware loop and executes all logic which controls movement through physics:

1. **Sensor Aggregation:** Identifies the right I2C multiplexer channel, gets the raw AS5600 encoder angle readings, and calculates the raw speed of rotation.
2. **Signal Filtering:** Filters out the raw RPM using a 5-level Median filter (for removing hardware anomalies), followed by a Dynamic Moving Average filter (the window of which is calculated depending on the speed of the vehicle).
3. **PI Controller Execution:** Takes in the RPM reading and compares it with the target RPM calculated by Core 1. Calculates the Proportional and Integral error terms and outputs the final PWM (-255 to +255).

3.2 First-Order System Modeling

Given that there will be differences between the motor manufacturers and the fact that the data sheet lacks complete specifications of the parameters, a DC motor can be considered a second-order system. However, a practical approach is to consider a first-order system. As the aim is tracking the commanded speed, then a basic PID or PI controller will be adequate for stable system operation. The magnetic encoder can effectively measure the position; however, due to the low sensor output frequency relative to the fast rotation of the shaft, skipping some angle measurements can happen. Thus, some filtering technique has been implemented to eliminate signal noise and ensure high accuracy; however, a cut-off frequency filter was not included due to a possible reduction in performance efficiency.[27]

$$G(s) = \frac{K}{\tau s + 1} \quad \text{Equation 3.2.2}$$

Where:

$$K = \frac{\text{Maximum Steady-State RPM}}{\text{Maximum PWM}} = 11.7$$

$$\tau = 0.9\text{s}$$

This indicates that the motor reaches 63.2% of steady-state. [25]

3.4 Step Response Analysis and Motor Control (PI Controller)

The system response confirms first-order behavior.

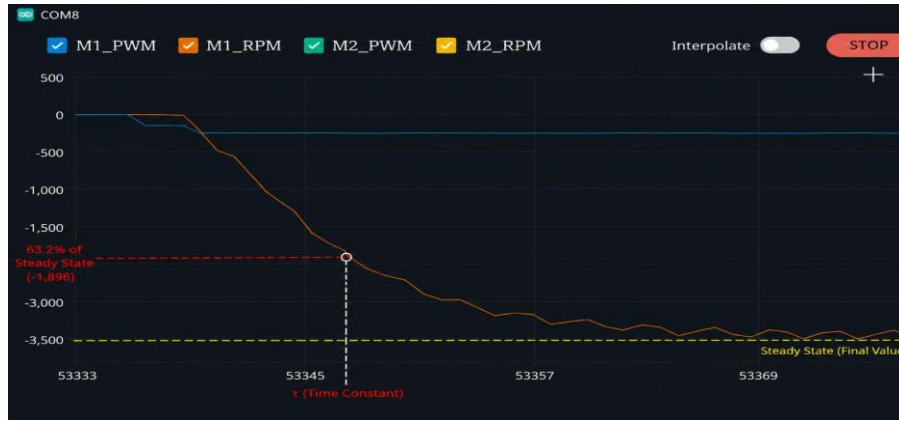


Figure 3.1: Step Response of DC Motor System

The system will use a classical PI controller:

$$u(t) = K_p e(t) + K_i \int e(t) dt \quad \text{Equation 2.3.4}$$

Discrete form:

$$u[k] = K_p e[k] + K_i \sum e[k] \Delta t \quad \text{Equation 2.3.5}$$

To get the K_p we use Equation 2.3.6 , $T_c = 0.5$ s so the result will be approximately 0.15

$$K_p = \frac{\tau}{K \times T_c} \quad \text{Equation 2.3.6}$$

And for K_i Equation 2.3.7 will be utilized to give approximately 0.12

$$K_i = \frac{K_p}{\tau} \quad \text{Equation 2.3.6}$$

By using the theoretical results, the internal parameters could be detected, as well as the system was put under experimental testing to see the response. However, the derivative term was not used because it amplified the encoder measurement noise, resulting in vibration and maximum speed fluctuations. The PI was sufficient to achieve a stable response and tracking performance. The final controller gains were experimentally tuned to ($K_p=0.6$) and ($K_i=0.4$), which gives a good response.

Appendix A: Calculations

1) Kinematic Position Analysis

- I. To solve for the outer steering angle (δ_o), we first define our given suspension parameters: $W = 164\text{mm}$, $L = 286.53\text{mm}$.

$$\delta_i = 30^\circ, \beta = 15.97^\circ, d = 22.76\text{mm}, L_T = 151.476\text{mm}$$

- II. The Four-Bar Linkage Distance Formula:

The relationship between the inner and outer steering angles is governed by the distance formula connecting the two pivot points via the tie rod.

$$\begin{aligned} \text{Tie Rod}^2 = & (w - d \sin(\beta + \delta_i) - d \sin(\beta - \delta_o))^2 \\ & + (d \cos(\beta - \delta_o) - d \cos(\beta + \delta_i))^2 \end{aligned}$$

- III. Substituting Known Values:

$$\begin{aligned} 151.476^2 = & (164.00 - 22.76 \sin(15.97^\circ + 30^\circ) \\ & - 22.76 \sin(15.97^\circ - \delta_o))^2 \\ & + (22.76 \cos(15.97^\circ - \delta_o) - 22.76 \cos(15.97^\circ + 30^\circ))^2 \end{aligned}$$

- IV. Simplifying Constants:

$$\begin{aligned} 22944.98 = & (164.00 - 22.76 \sin(45.97^\circ) - 22.76 \sin(15.97^\circ - \delta_o))^2 \\ & + (22.76 \cos(15.97^\circ - \delta_o) - 22.76 \cos(45.97^\circ))^2 \\ 22944.98 = & (147.64 - 22.76 \sin(15.97^\circ - \delta_o))^2 \\ & + (22.76 \cos(15.97^\circ - \delta_o) - 15.82)^2 \end{aligned}$$

- V. Expanding and Grouping Similar Terms:

$$-0.0565 = \sin(15.97^\circ - \delta_o) + 0.1071 \cos(15.97^\circ - \delta_o)$$

- VI. Final Solution:

$$\begin{aligned} 1.0057 \cdot \sin(15.97^\circ - \delta_o + 6.11^\circ) &= -0.0565 \\ \sin(22.08^\circ - \delta_o) &= -0.05618 \\ 22.08^\circ - \delta_o &= -3.22^\circ \\ \delta_o &= 22.08^\circ + 3.22^\circ = 25.3^\circ \end{aligned}$$

As seen from the calculation, if the inner angle is 30 deg, the outer will be approximately 25.3 deg, which is the approximate real-life relationship between both wheels for the chosen parameter of the 1/10 car scale. The Ideal Ackerman situation would yield to a different angle that makes all lines intersect at a point, the angle can be gotten as follows:

$$\tan(\delta_o) = \frac{l \cdot \tan(\delta_i)}{l + w \cdot \tan(\delta_i)}$$

$$\tan(\delta_o) = \frac{286.53 \cdot \tan(30^\circ)}{286.53 + 164.00 \cdot \tan(30^\circ)}$$

$$\tan(\delta_o) = \frac{286.53 \cdot 0.5774}{286.53 + 164.00 \cdot 0.5774}$$

$$\tan(\delta_o) = \frac{165.44}{286.53 + 94.69} = \frac{165.44}{381.22}$$

$$\tan(\delta_o) = 0.4340$$

$$\delta_o = \arctan(0.4340) = 23.46^\circ$$

Theoretical Outer Angle: 23.46 deg & **Actual Mechanical Linkage Outer Angle:** 25.3 deg. This brings a difference of 1.84 deg at the 30 deg position of the inner wheel and this difference is exactly what was plotted across the entire steering sweep to calculate the **RMS Error** of the mechanism.

Appendix B: The MATLAB Code

```
% Exact Geometric Steering Simulation: Safe Range Finder
clear; clc; close all;
%% =====
%% 1. DEFINE CONSTANTS
%% =====
W = 164; % Car Width (mm)
d = 22.76; % Steering arm length (mm)
beta = 15.97; % Initial bend angle / Ackermann angle (degrees)
Rs = 25; % Servo link length (mm)
```

```

% Define Stationary Servo Pivot Point (X5, Y5)
X5 = W / 2;
Y5 = Rs + d * cosd(beta);
% Calculate Constant Tie Rod Half-Length (at neutral beta_s = 0)
L_half = sqrt((W/2 - d*sind(beta))^2 + (d*cosd(beta) - d*cosd(beta))^2);
%% =====
%% 2. SWEEP TO FIND VALID OPERATING RANGE
%% =====
beta_s_test = -40:0.5:40; % Test a wide range with 0.5-degree resolution
valid_beta_s = [];
valid_delta1 = [];
valid_delta2 = [];
for bs = beta_s_test
% --- SERVO HORN TIP (X6, Y6) ---
X6 = W/2 + Rs * sind(bs);
Y6 = d * cosd(beta) + Rs * (1 - cosd(bs));
% --- CHECK LEFT ARM REACH ---
D1 = sqrt(X6^2 + Y6^2);
if D1 > d + L_half || D1 < abs(d - L_half)
continue; % Binding detected, skip this angle
end
% --- CHECK RIGHT ARM REACH ---
dx2 = X6 - W; dy2 = Y6 - 0;
D2 = sqrt(dx2^2 + dy2^2);
if D2 > d + L_half || D2 < abs(d - L_half)
continue; % Binding detected, skip this angle
end
% If it made it here, the geometry works! Calculate the coordinates.
a1 = (d^2 - L_half^2 + D1^2) / (2 * D1);
h1 = sqrt(max(0, d^2 - a1^2));
xm1 = a1 * X6 / D1; ym1 = a1 * Y6 / D1;
Y3a = ym1 - h1 * X6 / D1; Y3b = ym1 + h1 * X6 / D1;
if Y3a > 0; X3 = xm1 + h1 * Y6 / D1; Y3 = Y3a; else; X3 = xm1 - h1 * Y6 /
D1; Y3 = Y3b; end
a2 = (d^2 - L_half^2 + D2^2) / (2 * D2);
h2 = sqrt(max(0, d^2 - a2^2));
xm2 = W + a2 * dx2 / D2; ym2 = a2 * dy2 / D2;
Y4a = ym2 - h2 * dx2 / D2; Y4b = ym2 + h2 * dx2 / D2;
if Y4a > 0; X4 = xm2 + h2 * dy2 / D2; Y4 = Y4a; else; X4 = xm2 - h2 * dy2 /
D2; Y4 = Y4b; end
% Store the safe, valid data points
valid_beta_s(end+1) = bs;
valid_delta1(end+1) = atan2d(X3, Y3) - beta;
valid_delta2(end+1) = beta - atan2d(W - X4, Y4);
end
%% =====
%% 3. REPORT FINDINGS TO COMMAND WINDOW
%% =====
max_left = min(valid_beta_s);
max_right = max(valid_beta_s);
fprintf('\n===== \n');
fprintf(' GEOMETRY CAPABILITY REPORT \n');
fprintf('===== \n');
);

```

```

fprintf(' Steering Arm (d): %.2f mm\n', d);
fprintf(' Servo Horn (Rs): %.2f mm\n', Rs);
fprintf(' \n >>> MAXIMUM SAFE SERVO RANGE: %.1f deg to +%.1f deg <<<\n',
max_left, max_right);
fprintf('===== \n'
);
fprintf(' Servo Input (beta_s) | Left (Output: delta_1) | Right (Output:
delta_2)\n');
fprintf('-----
\n');
% Print table in 1-degree increments for readability
for i = 1:length(valid_beta_s)
if mod(valid_beta_s(i), 1) == 0 % Only print whole numbers to keep table
clean
fprintf(' %20.1f | %22.4f | %23.4f\n', valid_beta_s(i), valid_delta1(i),
valid_delta2(i));
end
end
fprintf('-----
\n');
%% =====
%% 4. GRAPHICAL SIMULATION (Animating only the safe range)
%% =====
fig = figure('Name', 'Safe Range Steering Simulation', 'Position', [100,
100, 900, 500]);
for i = 1:length(valid_beta_s)
bs = valid_beta_s(i);
X6 = W/2 + Rs * sind(bs);
Y6 = d * cosd(beta) + Rs * (1 - cosd(bs));
% Recalculate positions for plotting
D1 = sqrt(X6^2 + Y6^2);
a1 = (d^2 - L_half^2 + D1^2) / (2 * D1); h1 = sqrt(max(0, d^2 - a1^2));
xm1 = a1 * X6 / D1; ym1 = a1 * Y6 / D1;
Y3a = ym1 - h1 * X6 / D1; Y3b = ym1 + h1 * X6 / D1;
if Y3a > 0; X3 = xm1 + h1 * Y6 / D1; Y3 = Y3a; else; X3 = xm1 - h1 * Y6 /
D1; Y3 = Y3b; end
D2 = sqrt((X6-W)^2 + Y6^2);
a2 = (d^2 - L_half^2 + D2^2) / (2 * D2); h2 = sqrt(max(0, d^2 - a2^2));
xm2 = W + a2 * (X6-W) / D2; ym2 = a2 * Y6 / D2;
Y4a = ym2 - h2 * (X6-W) / D2; Y4b = ym2 + h2 * (X6-W) / D2;
if Y4a > 0; X4 = xm2 + h2 * Y6 / D2; Y4 = Y4a; else; X4 = xm2 - h2 * Y6 /
D2; Y4 = Y4b; end
clf; set(gca, 'YDir', 'reverse'); hold on; axis equal; grid on;
title(sprintf('Safe Operating Range | Servo (\\beta_s): %+.1f^\\circ | Left:
%+.1f^\\circ | Right: %+.1f^\\circ', ...
bs, valid_delta1(i), valid_delta2(i)));
xlabel('X-axis (mm)'); ylabel('Y-axis (mm) [Positive is Down]');
plot([0, W], [0, 0], 'k--', 'LineWidth', 1.5);
plot(0, 0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 8);
plot(W, 0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 8);
plot(X5, Y5, 'ks', 'MarkerFaceColor', 'k', 'MarkerSize', 8);
plot([0, X3], [0, Y3], 'b-o', 'LineWidth', 2);
plot([W, X4], [0, Y4], 'r-o', 'LineWidth', 2);
plot([X5, X6], [Y5, Y6], 'm-d', 'LineWidth', 2, 'MarkerFaceColor', 'm');
plot([X3, X6], [Y3, Y6], 'g-', 'LineWidth', 3);

```

```

plot([X4, X6], [Y4, Y6], 'g-', 'LineWidth', 3);
xlim([-40, W + 40]); ylim([-20, Y5 + 20]);
drawnow; pause(0.05);
end

```

Appendix C : MATLAB SURFACE OPTOMAZTIO 6-LINK MECHANISM

```

c close all
clear all
clc

%% 1. Constants & Geometry Setup
W = 164;    % Track width (mm)
L_c = 286.52; % Wheelbase (mm)
R_s = 25;    % Servo link length (mm)

% Stationary Pivot Points
X_1 = 0; Y_1 = 0; % KP1 (Left wheel pivot)
X_2 = W; Y_2 = 0; % KP2 (Right wheel pivot)
X_5 = W/2; Y_5 = 57; % Pivot point of the servo link

% Inputs for Optimization
servo_beta_deg = 0:0.5:35; % Servo input angles (degrees)
servo_beta_rad = deg2rad(servo_beta_deg);

% Wheel angle search space (used to find where the tie rod connects)
theta_vec_deg = 0:0.5:35;
theta_vec = deg2rad(theta_vec_deg);

% Optimization Ranges
alpha_range = 2:0.04:30; % Ackerman bend angle (degrees)
r_range = 10:0.04:30; % Steering arm length (mm)

rmse_surface = zeros(length(alpha_range), length(r_range));

%% 2. Coordinate-Based Optimization Loop
fprintf('Starting optimization...\n');

for a_idx = 1:length(alpha_range)
    alpha = deg2rad(alpha_range(a_idx));

    for r_idx = 1:length(r_range)
        r = r_range(r_idx);
        errors = zeros(1, numel(servo_beta_rad));
    end
end

```

```

% Weighting to prioritize the primary steering range
weights = exp(-0.5 * (servo_beta_deg/15).^2);

% Step A: Calculate exact Tie Rod length at neutral (beta = 0, theta = 0)
X_3_neutral = r * sin(alpha);
Y_3_neutral = r * cos(alpha);
X_6_neutral = W/2;
Y_6_neutral = 32; % From (57-25) + R_s(1 - cos(0))

L_T = sqrt((X_6_neutral - X_3_neutral)^2 + (Y_6_neutral - Y_3_neutral)^2);

% Step B: Sweep through servo inputs
for i = 1:numel(servo_beta_rad)
    beta = servo_beta_rad(i);

    % If driving straight, error is zero
    if beta == 0
        errors(i) = 0;
        continue;
    end

    % 1. Position of the Servo Link End (X_6, Y_6)
    X_6 = (W/2) + R_s * sin(beta);
    Y_6 = 32 + R_s * (1 - cos(beta));

    % 2. Solve for Left Wheel Angle (tita_t1)
    % X_3 = r*sin(alpha + theta), Y_3 = r*cos(alpha + theta)
    X_3_arr = r * sin(alpha + theta_vec);
    Y_3_arr = r * cos(alpha + theta_vec);
    dist_L = sqrt((X_6 - X_3_arr).^2 + (Y_6 - Y_3_arr).^2);
    [~, idx_L] = min(abs(dist_L - L_T)); % Find angle that matches tie rod length
    tita_t1 = theta_vec(idx_L);

    % 3. Solve for Right Wheel Angle (tita_t2)
    % X_4 = W - r*sin(alpha - theta), Y_4 = r*cos(alpha - theta)
    X_4_arr = W - r * sin(alpha - theta_vec);
    Y_4_arr = r * cos(alpha - theta_vec);
    dist_R = sqrt((X_6 - X_4_arr).^2 + (Y_6 - Y_4_arr).^2);
    [~, idx_R] = min(abs(dist_R - L_T));
    tita_t2 = theta_vec(idx_R);

    % 4. Determine Inner and Outer Wheels
    % The wheel turned at a sharper angle is the inner wheel
    if abs(tita_t1) > abs(tita_t2)
        theta_inner = abs(tita_t1);
    end
end

```

```

        theta_outer_actual = abs(tita_t2);
    else
        theta_inner = abs(tita_t2);
        theta_outer_actual = abs(tita_t1);
    end

    % 5. Calculate Ideal Ackermann Outer Angle
    theta_outer_ideal = acot(W/L_c + cot(theta_inner));

    % 6. Calculate Error
    errors(i) = abs(theta_outer_ideal - theta_outer_actual) * weights(i);
end
rmse_surface(a_idx, r_idx) = sqrt(mean(errors.^2));
end
end

%%% 3. Finding the "Real" Optimal
[min_err, min_idx] = min(rmse_surface(:));
[best_a_idx, best_r_idx] = ind2sub(size(rmse_surface), min_idx);
opt_alpha = alpha_range(best_a_idx);
opt_r = r_range(best_r_idx);

fprintf('--- True Angle Optimization Results ---\n');
fprintf('Best Alpha (Initial Bend): %.2f deg | Best Arm Length (r): %.2f mm\n',
    opt_alpha, opt_r);

%%% 4. Plotting
[R_mesh, A_mesh] = meshgrid(r_range, alpha_range);
figure(1);
surf(R_mesh, A_mesh, rmse_surface);
shading interp; colormap jet; colorbar;
xlabel('Steering Arm Length(mm)');
ylabel('band angle (deg)');
zlabel('Angle Errors (RMSE)');
title('6link Error Surface');

% Plot Comparison for the Optimal Parameters
opt_alpha_rad = opt_alpha * (pi/180);
X_3_n = opt_r * sin(opt_alpha_rad);
Y_3_n = opt_r * cos(opt_alpha_rad);
L_T_opt = sqrt((W/2 - X_3_n)^2 + (32 - Y_3_n)^2);

actual_inner = zeros(1, numel(servo_beta_rad));
actual_outer = zeros(1, numel(servo_beta_rad));
ideal_outer = zeros(1, numel(servo_beta_rad));

```

```

for i = 1:numel(servo_beta_rad)
    beta = servo_beta_rad(i);
    if beta == 0
        continue;
    end

    X_6 = (W/2) + R_s * sin(beta);
    Y_6 = 32 + R_s * (1 - cos(beta));

    % Left
    X_3_arr = opt_r * sin(opt_alpha_rad + theta_vec);
    Y_3_arr = opt_r * cos(opt_alpha_rad + theta_vec);
    [~, idx_L] = min(abs(sqrt((X_6 - X_3_arr).^2 + (Y_6 - Y_3_arr).^2) - L_T_opt));
    t1 = theta_vec(idx_L);

    % Right
    X_4_arr = W - opt_r * sin(opt_alpha_rad - theta_vec);
    Y_4_arr = opt_r * cos(opt_alpha_rad - theta_vec);
    [~, idx_R] = min(abs(sqrt((X_6 - X_4_arr).^2 + (Y_6 - Y_4_arr).^2) - L_T_opt));
    t2 = theta_vec(idx_R);

    if abs(t1) > abs(t2)
        actual_inner(i) = abs(t1);
        actual_outer(i) = abs(t2);
    else
        actual_inner(i) = abs(t2);
        actual_outer(i) = abs(t1);
    end
    ideal_outer(i) = acot(W/L_c + cot(actual_inner(i)));
end

figure(2);
plot(ideal_outer, actual_inner, 'r', 'LineWidth', 2); hold on; grid on;
plot(actual_outer, actual_inner, 'b--', 'LineWidth', 1.5);
legend('Ideal Ackermann', ['Optimized r=' num2str(opt_r) ' \alpha='
num2str(opt_alpha)]);
xlabel('Outer Wheel Angle (rad)');
ylabel('Inner Wheel Angle (rad)');
title('Matching the Curves via Coordinate Geometry');

```

Appendix D : EMBEDDED CODE

```

#include <Arduino.h>
#include <ESP32Servo.h>
#include <Wire.h>

```

```

#include <AS5600.h>

// =====
// --- DYNAMIC AVERAGE FILTER ---
// =====

constexpr int MEDIAN_WINDOW = 5;
constexpr int MAX_DYN_MEAN_WINDOW = 25;

// Structure to define the speed range and its corresponding filter window size
struct FilterRange {
    float maxThreshold; // Maximum RPM for this specific range
    int windowSize;     // Number of samples to average (filter strength)
};

// Define the ranges (adjust these values based on your motor's actual behavior)
const FilterRange rpmRanges[10] = {
    {300.0f, 4}, {600.0f, 6}, {900.0f, 8}, {1200.0f, 10},
    {1500.0f, 14}, {1800.0f, 14}, {2100.0f, 16}, {2400.0f, 18},
    {2700.0f, 20}, {3000.0f, 25}
};

// =====
// 1. PIN DEFINITIONS & I2C
// =====

constexpr uint8_t RPWM1    = 19;
constexpr uint8_t LPWM1    = 23;
constexpr uint8_t RPWM2    = 4;
constexpr uint8_t LPWM2    = 18;
constexpr uint8_t SERVO_PIN = 14;
constexpr uint8_t CH1_PIN  = 15;
constexpr uint8_t CH2_PIN  = 13;
constexpr uint8_t TCA_ADDR = 0x70;

// =====
// 2. DUAL CORE THREAD SAFETY (MUTEX)
// =====

// targetMux ONLY protects targetRPM_L / targetRPM_R
// (written on Core1 via applyAckermann, read on Core0 in calculatePI)
portMUX_TYPE targetMux = portMUX_INITIALIZER_UNLOCKED;
portMUX_TYPE rcMux     = portMUX_INITIALIZER_UNLOCKED; // RC raw
values
TaskHandle_t Core0TaskHandle;

// =====
// 3. STEERING & SERVO SYSTEM
// =====

```



```

struct SteeringSystem {
    Servo  object;
    const int minDegrees = 60;
    const int maxDegrees = 120;
    float currentAngle = 90.0f; // 90 = center/straight
};
SteeringSystem mySteering;

// =====
// 4. MOTOR & PI CONTROL STRUCTURE
// =====
// [F6] targetRPM is now in ACTUAL RPM units.
// Mapping: RC [-2800..2800] PWM-equiv → [-MAX_RPM..MAX_RPM]
// Adjust MAX_RPM to match your motor datasheet.
constexpr float MAX_RPM = 2800.0f; // ← Set to your motor's measured max RPM

struct MotorSystem {
    const int id;
    const float Kp = 0.55f;
    const float Ki = 0.4f;

    int prevAngle = 0;
    float integralSum = 0.0f;
    float calcOutput = 0.0f; // PWM output [-255..255]

    // Cross-core shared — protected by targetMux
    volatile float targetRPM = 0.0f;
    // Core-0 internal only — NO mutex needed
    float measuredRPM = 0.0f;

    // [F11] I2C error counter
    uint8_t i2cErrorCount = 0;

    // --- Filter Memory Buffers ---
    int medianBuffer[MEDIAN_WINDOW] = {0};
    int medianIndex = 0;

    float meanBuffer[MAX_DYN_MEAN_WINDOW] = {0.0f};
    int meanIndex = 0;
};

MotorSystem motorL = {1};
MotorSystem motorR = {2};

AS5600 encoderL;
AS5600 encoderR;

```

```

// =====
// 5. ACKERMANN GEOMETRY
// =====
struct AckermannChassis {
    const float L = 286.52f; // Wheelbase (mm)
    const float w = 82.0f;  // Track width (mm)
};
AckermannChassis chassis;

// =====
// 6. RC CHANNELS
// =====
struct RCChannel {
    const uint8_t pin;
    volatile unsigned long startTime = 0;
    volatile uint32_t rawValue = 1500;
    float filteredValue = 1500.0f;
};
constexpr float FILTER_WEIGHT = 0.1f;
RCChannel steerCh = {CH1_PIN};
RCChannel speedCh = {CH2_PIN};

// =====
// 7. NON-BLOCKING ULTRASONIC [F4] [US-FIX]
// =====
struct UltrasonicSensor {
    const uint8_t trigPin;
    const uint8_t echoPin;
    int distanceCm = 999;
    int prevDistanceCm = 999;
    int obstacleCount = 0;
    unsigned long lastObstacleTime = 0;
};

UltrasonicSensor adas[4] = {
    {25, 33}, // Front — index 0
    {5, 5},  // Back  — index 1
    {27, 26}, // Left  — index 2
    {16, 17} // Right — index 3
};

unsigned long lastSensorCheck = 0;

// =====
// 8. CAR STATE MACHINE

```

```

// =====
enum CarState { NORMAL_RC, AVOIDANCE_OVERRIDE,
BRAKING_SEQUENCE, SYSTEM_HALTED };

struct StateManager {
    CarState    currentState    = NORMAL_RC;
    unsigned long stateTimer    = 0;
    int         overrideSpeed   = 0;
    float       overrideSteering = 0.0f;
    float       speedBeforeBraking = 0.0f; // [F10] float, not int
};
StateManager carControl;

// =====
// --- HELPER: TCA9548A I2C MUX SELECT ---
// =====
void tcaSelect(uint8_t channel) {
    if (channel > 7) return;
    Wire.beginTransaction(TCA_ADDR);
    Wire.write(1 << channel);
    Wire.endTransmission();
}

// =====
// --- MOTOR DRIVER OUTPUT ---
// =====
void setMotorBTS(float speed, uint8_t rpwm_pin, uint8_t lpwm_pin) {
    int pwm = (int)constrain(speed, -255.0f, 255.0f);
    if (pwm > 0) {
        analogWrite(rpwm_pin, pwm);
        analogWrite(lpwm_pin, 0);
    } else if (pwm < 0) {
        analogWrite(rpwm_pin, 0);
        analogWrite(lpwm_pin, -pwm);
    } else {
        analogWrite(rpwm_pin, 0);
        analogWrite(lpwm_pin, 0);
    }
}

// =====
// --- RC INTERRUPTS ---
// =====
void IRAM_ATTR readRCChannel(void* arg) {
    RCChannel* ch = static_cast<RCChannel*>(arg);
    if (digitalRead(ch->pin) == HIGH) {

```

```

        ch->startTime = micros();
    } else {
        portENTER_CRITICAL_ISR(&rcMux);
        ch->rawValue = micros() - ch->startTime;
        portEXIT_CRITICAL_ISR(&rcMux);
    }
}

void setupRCInputPins() {
    pinMode(steerCh.pin, INPUT_PULLDOWN);
    pinMode(speedCh.pin, INPUT_PULLDOWN);
    attachInterruptArg(digitalPinToInterrupt(steerCh.pin), readRCChannel, &steerCh,
CHANGE);
    attachInterruptArg(digitalPinToInterrupt(speedCh.pin), readRCChannel, &speedCh,
CHANGE);
}

void getSafeRCValues(int &steer, int &speed) {
    portENTER_CRITICAL(&rcMux);
    steer = steerCh.rawValue;
    speed = speedCh.rawValue;
    portEXIT_CRITICAL(&rcMux);
    if (steer < 900 || steer > 2100) steer = 1500;
    if (speed < 900 || speed > 2100) speed = 1500;
}

void applyRCSmoothingFilter(RCChannel &ch, int safeValue) {
    ch.filteredValue = (FILTER_WEIGHT * safeValue) + ((1.0f - FILTER_WEIGHT) *
ch.filteredValue);
}

// =====
// --- ENCODER READING [F9, F11] ---
// =====

int readAS5600Angle(AS5600 &enc) {
    uint16_t raw = enc.readAngle();
    return (int)raw;
}

void processEncoder(MotorSystem &motor, AS5600 &enc, float deltaT) {
    tcaSelect(motor.id - 1);
    int currAngle = readAS5600Angle(enc);

    if (currAngle < 0) {
        motor.i2cErrorCount++;
        // [F11] Re-initialise bus after 5 consecutive errors
    }
}

```

```

    if (motor.i2cErrorCount >= 5) {
        Wire.end();
        delay(2);
        Wire.begin();
        Wire.setClock(400000);
        enc.begin();
        motor.i2cErrorCount = 0;
        Serial.println("[I2C] Bus recovered.");
    }
    return;
}

motor.i2cErrorCount = 0;

int delta = currAngle - motor.prevAngle;
if (delta < -2048) delta += 4096;
else if (delta > 2048) delta -= 4096;

float velocity = (float)delta / deltaT;
float rawRPM = (velocity / 4096.0f) * 60.0f;
if (motor.id == 2) {
    rawRPM = -rawRPM;
}
// -----

// --- A) MEDIAN FILTER ---
motor.medianBuffer[motor.medianIndex] = (int)rawRPM;

// --- A) MEDIAN FILTER ---
motor.medianBuffer[motor.medianIndex] = (int)rawRPM;
motor.medianIndex = (motor.medianIndex + 1) % MEDIAN_WINDOW;

int tempSort[MEDIAN_WINDOW];
for (int i = 0; i < MEDIAN_WINDOW; i++) tempSort[i] = motor.medianBuffer[i];

// Inline Bubble Sort (fast for N=5)
for (int i = 0; i < MEDIAN_WINDOW - 1; i++) {
    for (int j = i + 1; j < MEDIAN_WINDOW; j++) {
        if (tempSort[i] > tempSort[j]) {
            int temp = tempSort[i]; tempSort[i] = tempSort[j]; tempSort[j] = temp;
        }
    }
}
float medianCleanValue = (float)tempSort[MEDIAN_WINDOW / 2];

// --- B) 10-STAGE DYNAMIC MEAN FILTER ---
motor.meanBuffer[motor.meanIndex] = medianCleanValue;

```

```

motor.meanIndex = (motor.meanIndex + 1) % MAX_DYN_MEAN_WINDOW;

float absVal = fabsf(medianCleanValue);
int activeWindow = rpmRanges[9].windowSize; // Default to max size

for (int i = 0; i < 10; i++) {
    if (absVal <= rpmRanges[i].maxThreshold) {
        activeWindow = rpmRanges[i].windowSize;
        break;
    }
}

float sum = 0.0f;
for (int i = 0; i < activeWindow; i++) {
    int lookbackIndex = (motor.meanIndex - 1 - i + MAX_DYN_MEAN_WINDOW)
% MAX_DYN_MEAN_WINDOW;
    sum += motor.meanBuffer[lookbackIndex];
}

motor.measuredRPM = sum / (float)activeWindow;
motor.prevAngle = currAngle;
}

// =====
// --- PI CONTROLLER [F2, F6] ---
// =====

void calculatePI(MotorSystem &motor, float deltaT) {
    // [F2] Only lock what is needed: reading the cross-core targetRPM
    float target;
    portENTER_CRITICAL(&targetMux);
    target = motor.targetRPM;
    portEXIT_CRITICAL(&targetMux);

    if (carControl.currentState != NORMAL_RC) {
        motor.integralSum = 0.0f;
    }

    if (target == 0.0f) {
        motor.calcOutput = 0.0f;
        motor.integralSum = 0.0f;
        return;
    }

    float error = target - motor.measuredRPM;
    float P = motor.Kp * error;

```

```

    motor.integralSum += error * deltaT;
    motor.integralSum = constrain(motor.integralSum, -500.0f, 500.0f);
    float I = motor.Ki * motor.integralSum;

    motor.calcOutput = constrain(P + I, -255.0f, 255.0f);
}

// =====
// --- STEERING ---
// =====
void updateSteering(SteeringSystem &steering, RCChannel &rc) {
    int mappedAngle = (int)map((long)rc.filteredValue, 1000, 2000, -26, 27);
    if (abs(mappedAngle) < 1) mappedAngle = 0; // Deadband
    int finalAngle = constrain(93 + mappedAngle, steering.minDegrees,
    steering.maxDegrees);/////////93,87

    if (carControl.currentState == NORMAL_RC) {
        steering.object.write(finalAngle);
        steering.currentAngle = (float)finalAngle;
    }
}

// =====
// --- ACKERMANN [F5] ---
// =====
void applyAckermann(float baseRPM, float theta_deg) {
    float speedLeft = baseRPM;
    float speedRight = baseRPM;

    if (fabsf(theta_deg) >= 1.0f) {
        float theta_rad = theta_deg * DEG_TO_RAD;
        float R = fabsf(chassis.L / tanf(theta_rad));

        float R_outer = R + (chassis.w / 2.0f);
        float R_inner = R - (chassis.w / 2.0f);
        float ratio = (R_inner > 0.0f) ? (R_inner / R_outer) : 0.0f;
        float speed_inner = baseRPM * ratio;

        if (theta_deg > 0.0f) { // Right turn
            speedLeft = baseRPM;
            speedRight = speed_inner;
        } else { // Left turn
            speedLeft = speed_inner;
            speedRight = baseRPM;
        }
    }
}

```

```

    portENTER_CRITICAL(&targetMux);
    motorL.targetRPM = speedLeft;
    motorR.targetRPM = speedRight;
    portEXIT_CRITICAL(&targetMux);
}

// =====
// SONAR [F4] [US-FIX] ---
// =====

int getDistance(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    const unsigned long TIMEOUT = 4000;
    unsigned long startWait = micros();

    while (digitalRead(echoPin) == LOW) {
        if (micros() - startWait > TIMEOUT) return 999;
    }

    unsigned long pulseStart = micros();

    while (digitalRead(echoPin) == HIGH) {
        if (micros() - pulseStart > TIMEOUT) return 999;
    }

    unsigned long pulseEnd = micros();

    long duration = pulseEnd - pulseStart;

    int distance = duration * 0.034 / 2;

    if (distance <= 0 || distance > 100) return 999;

    return distance;
}

// =====
// --- OBSTACLE AVOIDANCE [F7] [US-FIX] ---
// =====
void checkObstacles() {

```



```

if (millis() - lastSensorCheck >= 30) {
    lastSensorCheck = millis();

    static int scanStep = 0;
    int sensorIndex = 0;

    // =====
    // Priority Scheduling
    // =====
    if (scanStep == 0 || scanStep == 2 || scanStep == 4) {
        sensorIndex = 0;
    } else if (scanStep == 1) {
        sensorIndex = 2;
    } else if (scanStep == 3) {
        sensorIndex = 3;
    } else if (scanStep == 5) {
        sensorIndex = 1;
    }

    scanStep = (scanStep + 1) % 6;

    adas[sensorIndex].distanceCm = getDistance(adas[sensorIndex].trigPin,
    adas[sensorIndex].echoPin);
    int distance = adas[sensorIndex].distanceCm;

    // =====
    // 1.
    // =====
    if (sensorIndex == 0) {
        if (distance >= 4 && distance <= 45) {
            adas[0].obstacleCount++;
        } else {
            adas[0].obstacleCount = 0;
        }

        if (carControl.currentState == NORMAL_RC) {
            if (adas[0].obstacleCount == 2) {
                Serial.print("[ADAS] FRONT Obstacle Confirmed at: ");
                Serial.println(distance);
                carControl.currentState = AVOIDANCE_OVERRIDE;
                carControl.stateTimer = millis();
                carControl.overrideSpeed = 500;
                carControl.overrideSteering = 0;
            }
            else if (adas[0].obstacleCount >= 4) {
                Serial.println("[ADAS] FRONT Emergency Braking Sequence!");
            }
        }
    }
}

```

```

        carControl.currentState = BRAKING_SEQUENCE;
        carControl.stateTimer = millis();
        carControl.speedBeforeBraking = motorL.measuredRPM;
    }
}
else if (carControl.currentState == AVOIDANCE_OVERRIDE) {
    if (carControl.overrideSpeed < 0 && adas[0].obstacleCount >= 2) {
        Serial.println("[ADAS] Front blocked during evasion! Reversing.");
        carControl.overrideSpeed = 500;
        if (carControl.overrideSteering > 0) carControl.overrideSteering = -15;
        else if (carControl.overrideSteering < 0) carControl.overrideSteering = 15;
        carControl.stateTimer = millis();
        adas[0].obstacleCount = 0;
    }
}
}

// =====
// 2.
// =====
else {
    if (distance >= 3 && distance <= 20) {
        adas[sensorIndex].obstacleCount++;

        if (carControl.currentState == NORMAL_RC &&
adas[sensorIndex].obstacleCount == 1) {
            Serial.print("[ADAS] Side/Back Triggered! Sensor: ");
Serial.println(sensorIndex);
            carControl.currentState = AVOIDANCE_OVERRIDE;
            carControl.stateTimer = millis();

            if (sensorIndex == 1) { carControl.overrideSpeed = -500;
carControl.overrideSteering = 0; }
            if (sensorIndex == 2) { carControl.overrideSpeed = -500;
carControl.overrideSteering = 15; }
            if (sensorIndex == 3) { carControl.overrideSpeed = -500;
carControl.overrideSteering = -15; }
        }
    } else {
        adas[sensorIndex].obstacleCount = 0;
    }
}
}
}

// =====

```

```

// --- ADAS STATE MACHINE [F3] ---
// =====
void applyADASLogic(float &baseRPM, float &steeringAngle) {
    unsigned long elapsed = millis() - carControl.stateTimer;

    switch (carControl.currentState) {
        case NORMAL_RC: break;

        case AVOIDANCE_OVERRIDE:
            if (elapsed <= 700) {
                baseRPM = carControl.overrideSpeed;
                steeringAngle = carControl.overrideSteering;
                mySteering.object.write(93 + (int)steeringAngle);
                mySteering.currentAngle = 93.0f + steeringAngle;

            } else {
                Serial.println("[ADAS] Evasive Action Complete.");
                carControl.currentState = NORMAL_RC;
            }
            break;

        case BRAKING_SEQUENCE:
            steeringAngle = 0;
            mySteering.object.write(93);
            mySteering.currentAngle = 93.0f;
            if (elapsed < 666) baseRPM = carControl.speedBeforeBraking * 0.66;
            else if (elapsed < 1333) baseRPM = carControl.speedBeforeBraking * 0.33;
            else if (elapsed < 2000) baseRPM = 0;
            else if (elapsed < 5000) baseRPM = 0;
            else {
                Serial.println("[ADAS] SYSTEM HALTED.");
                carControl.currentState = SYSTEM_HALTED;
            }
            break;

        case SYSTEM_HALTED:
            if (steerCh.filteredValue <= 1050 && speedCh.filteredValue <= 1050) {
                Serial.println("[ADAS] System Revived by Transmitter!");
                carControl.currentState = NORMAL_RC;
                for(int i = 0; i < 4; i++) {
                    adas[i].obstacleCount = 0;
                    adas[i].lastObstacleTime = 0;
                }
            } else {
                baseRPM = 0;
                steeringAngle = 0;
            }
    }
}

```

```

    }
    break;
}
}

// =====
// --- DRIVE KINEMATICS ---
// =====
void updateDriveKinematics() {
    int rcVal = (int)speedCh.filteredValue;
    float baseRPM = map(rcVal, 1000, 2000, (int)(-MAX_RPM), (int)(MAX_RPM));
    if (fabsf(baseRPM) < (MAX_RPM * 0.05f)) baseRPM = 0.0f; // 5% deadband

    float theta_deg = mySteering.currentAngle - 93.0f; // [F5] offset from center
    applyADASLogic(baseRPM, theta_deg);
    applyAckermann(baseRPM, theta_deg);
}

// =====
// --- TELEMETRY ---
// =====
void printDebuggingTelemetry() {
    static unsigned long lastPrint = 0;
    if (millis() - lastPrint < 100) return; // Rate-limit to 10 Hz
    lastPrint = millis();

    Serial.print("Tgt_L:"); Serial.print(motorL.targetRPM, 1);
    Serial.print(",Meas_L:"); Serial.print(motorL.measuredRPM, 1);
    Serial.print("Tgt_R:"); Serial.print(motorR.targetRPM, 1);
    Serial.print(",Meas_R:"); Serial.print(motorR.measuredRPM, 1);
    Serial.print(",State:"); Serial.print((int)carControl.currentState);
    Serial.print(",Steer:"); Serial.println(mySteering.currentAngle - 90.0f, 1);
}

// =====
// --- CORE 0 TASK [F2, F8] ---
// =====
void Core0Task_Manager(void* pvParameters) {
    unsigned long taskPrevT = micros();

    for (;;) {
        unsigned long currT = micros();
        float deltaT = (float)(currT - taskPrevT) / 1.0e6f;

        if (deltaT >= 0.001f) { // [F8] 1 ms guard
            processEncoder(motorL, encoderL, deltaT);

```

```

        processEncoder(motorR, encoderR, deltaT);

        calculatePI(motorL, deltaT); // [F2] mutex only inside calculatePI
        calculatePI(motorR, deltaT);

        if (carControl.currentState == SYSTEM_HALTED) {
            setMotorBTS(0, RPWM1, LPWM1);
            setMotorBTS(0, RPWM2, LPWM2);
        } else {
            setMotorBTS(motorL.calcOutput, RPWM1, LPWM1);
            setMotorBTS(motorR.calcOutput, RPWM2, LPWM2);
        }
        taskPrevT = currT;
    }
    vTaskDelay(1 / portTICK_PERIOD_MS); // [F8] 1 ms matches deltaT guard
}
}

// =====
// --- SETUP ---
// =====
void setup() {
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(400000);

    setupRCInputPins();
    mySteering.object.attach(SERVO_PIN);

    pinMode(RPWM1, OUTPUT); pinMode(LPWM1, OUTPUT);
    pinMode(RPWM2, OUTPUT); pinMode(LPWM2, OUTPUT);
    analogWriteFrequency(RPWM1, 20000); analogWriteFrequency(LPWM1, 20000);
    analogWriteFrequency(RPWM2, 20000); analogWriteFrequency(LPWM2, 20000);

    // Sonar pins + non-blocking echo ISRs [F4]
    for (int i = 0; i < 4; i++) {
        pinMode(adas[i].trigPin, OUTPUT);
        pinMode(adas[i].echoPin, INPUT);
    }

    // [F9] Initialise both encoder instances via TCA channels
    tcaSelect(0); encoderL.begin(); motorL.prevAngle = (int)encoderL.readAngle();
    tcaSelect(1); encoderR.begin(); motorR.prevAngle = (int)encoderR.readAngle();

    Serial.println("[INIT] System Ready");
}

```

```

xTaskCreatePinnedToCore(
    Core0Task_Manager,
    "Core0_Motor",
    10000,
    NULL,
    2,
    &Core0TaskHandle,
    0
);
}

// =====
// --- MAIN LOOP (CORE 1) ---
// =====
void loop() {

    int current_steer, current_speed;
    getSafeRCValues(current_steer, current_speed);

    applyRCSmoothingFilter(steerCh, current_steer);
    applyRCSmoothingFilter(speedCh, current_speed);

    checkObstacles();      // Non-blocking sonar round-robin [F4][US-FIX]
    updateSteering(mySteering, steerCh);
    updateDriveKinematics(); // Sends RPM targets to Core0 via mutex
    printDebuggingTelemetry(); // Rate-limited to 10 Hz internally

    vTaskDelay(40 / portTICK_PERIOD_MS);
}
}

```

References

- [1] J. Nocedal and S. J. Wright, Numerical optimization. Springer, 2006.
- [2] D. C. Montgomery, Design and analysis of experiments. John wiley & sons, 2017.
- [3] T. Chai, R. R. Draxler, et al., “Root mean square error (rmse) or mean absolute error (mae),” Geoscientific model development discussions, vol. 7, no. 1, pp. 1525–1534, 2014.12
- [4] T. Gillespie, Fundamentals of vehicle dynamics. SAE international, 2021.
- [5] F. Merat, “Introduction to robotics: Mechanics and control,” IEEE Journal on Robotics and Automation, vol. 3, no. 2, pp. 166–166, 1987.

- [6] K. Yaw-Hong and P. Da-Chen, “Dimensional synthesis and optimization of leading and mixed-leading double four-bar steering mechanisms: A comparative metaheuristic approach,” *Machines*, vol. 14, no. 4, p. 445, 2026.
- [7] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of machine learning research*, vol. 13, no. 2, 2012.
- [8] R. N. Jazar, *Vehicle dynamics: theory and application*. Springer Nature, 2025.
- [9] A. P. Stoicescu, “On the optimization of an ackermann steering linkage,” *UPB Scientific Bulletin, Series D: Mech Engi*, vol. 75, no. 4, pp. 5–9, 2013.
- [10] A. R. Hanzaki, P. Rao, and S. Saha, “Kinematic and sensitivity analysis and optimization of planar rack-and-pinion steering linkages,” *Mechanism and Machine theory*, vol. 44, no. 1, pp. 42–56, 2009.
- [11] D. Bastow, “Steering mechanisms,” *Proceedings of the Institution of Mechanical Engineers: Automobile Division*, vol. 184, no. 1, pp. 137–156, 1969.
- [12] J.-S. Zhao, X. Liu, Z.-J. Feng, and J. S. Dai, “Design of an ackermann-type steering mechanism,” *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 227, no. 11, pp. 2549–2562, 2013.
- [13] D. Pradhan, K. Ganguly, B. Swain, and H. Roy, “Optimal kinematic synthesis of 6 bar rack and pinion ackerman steering linkage,” *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, vol. 235, no. 6, pp. 1660–1669, 2021.
- [14] Y.-h. Hu, H. Dong, and J.-a. Zhang, “Calculation and optimization of the optimal region value of integral trapezoidal steering,” *Journal of Highway and Transportation Research and Development (English Edition)*, vol. 13, no. 2, pp. 103–110, 2019.
- [15] F. Dellaert and S. Hutchinson, “7.2. multi-rotor aircraft — introduction to robotics and perception.” https://www.roboticsbook.org/S70_drone_intro.html, n.d.
- [16] C.-L. Chiang and R.-F. Fung, “Mechanical design and motion analysis of

a pick-and-place mechanism,” Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, vol. 222, no. 12, pp. 2463–2473, 2008.

[17] E. A. N. Here, “Schematics of trapezoidal steering mechanism.” Research-Gate, Enter Year Here. Accessed: 2026-05-10.

[18] AS5600 Magnetic Rotary Position Sensor, *AS5600 Magnetic Rotary Position Sensor Datasheet*.

[19] HC-SR04 Ultrasonic Sensor, *HC-SR04 Ultrasonic Sensor Datasheet*.

[20] Robotistan, *Tower Pro MG996R High Torque Servo Motor 13kg 180 Degree*, 2026.

[Online]. Available: <https://www.robotistan.com/tower-pro-mg996r-high-torque-servo-motor-13kg-180-degree>. Accessed: May 27, 2026..

[21] TCA9548A I²C Multiplexer, *TCA9548A Low-Voltage 8-Channel I²C Switch Datasheet*.

[22] LM2596 DC-DC Converter, *LM2596 DC-DC Converter Datasheet*.

[23] FS-iA6B Receiver, *FS-iA6B Receiver Datasheet*.

[24] Robitshop, *RS-775 DC Motor Rulmanlı 12V 3000 RPM Şarjlı Matkap Motoru*, 2026.

[Online]. Available: <https://www.robitshop.com/urun/rs-775-dc-motor-rulmanli-12v-3000-rpm-sarjli-matkap-motoru>. Accessed: May 27, 2026.

[25] K. Ogata, *Modern Control Engineering*, 2010.

[26] NXP, *I²C Bus Specification UM10204*, 2021.

[27] Infineon Technologies AG, *BTS7960 High Current PN Half Bridge NovalithIC – Datasheet*, Rev. 1.1, Dec. 2004. [Online]. Available: <https://www.infineon.com/>. Accessed: May 27, 2026.

[28] Analog Devices, *MT-101: Decoupling, Filtering, and Bypassing in High-Speed Digital Systems*, Analog Devices, 2009. [Online]. Available: <https://www.analog.com/media/en/training-seminars/tutorials/MT-101.pdf>. Accessed: May 27, 2026.